实验要回答什么：两次实现变化，分别验证

问题 A：保存路径

同步保存 → 异步保存

完整文件是否更早就绪？
主流程负担转移到哪里？
质量检查是否仍通过？

主要证据：v1 四配置对照

问题 B：规划后端

v1 auto → v2 auto

接口怎样迁移？
能否完整生成并校验？
是否建立性能优势？

主要证据：v2 新重跑

讲解顺序

先交代实验与计时口径
再说明实现机制
然后读效率与内部耗时
最后讨论质量和局限

主线 45 页，附录 6 页

当前材料：v1 保存方式对照、历史保存实验与 v2 新重跑，均使用已有记录。

支持结论：先分清保存路径改造与后端迁移两个问题，再分别评估效率与质量。

读图讲解：先讲两个问题：保存优化是否有效，规划后端迁移是否另有收益。

实验矩阵：哪些条件相同，哪些条件不同

记录组	条件 / 范围	改变因素	样本与验收
v1 同步	10/14 · 8 环境 · seed 101	同步保存；0 writer	17 完成 / 16 保存 / 16 末态通过
v1 cap4 / cap8 / auto	10/14 · 8 环境 · seed 101	异步策略不同；实际均 4 writer	各 17 完成 / 16 保存 / 16 通过
v2 auto 重跑	10/15 · 8 环境 · seed 101	规划后端变化；实际 4 writer	16 完成 / 16 保存 / 15 通过
早期生产 / 离线保存	24 条生产 / 4 条离线	同步、单 writer、双 writer	不同批次；只作补充证据
离线压缩对照	1342 帧 / 相机；固定 chunk	gzip / none / lzf	仅比较保存耗时与体积

当前实验：v1 四配置各一轮；v2 auto 次日一轮；历史与离线实验单列。

支持结论：生产、离线与跨日期记录的比较范围不同，不能合并成同一统计样本。

读图讲解：先看共同条件，再看改变因素；配置上限和实际 writer 数分别读取。

指标口径：同一条数据，可以有不同的完成时刻

整批墙钟时间

T = 首条开始 → output_ready
包含生成与最终收尾
不含首条开始前全部启动

完整启动退出用 launcher 时间
不要混用两个计时边界

吞吐与质量分母

记录成功吞吐 = 16 × 60 / T
末态合格吞吐 = 通过数 × 60 / T

v1：末态通过 16 条
v2：末态通过 15 条
不能把保存成功直接当质量合格

内部阶段计时

v1：17 个完成 episode 的均值
v2：16 个完成 episode 的均值
后台：每组 16 次保存事件

并发 episode 的耗时重叠
阶段均值之和 ≠ 整批墙钟

当前口径：首条开始至 output_ready 计最终就绪时间，阶段均值按完成 episode 计。

支持结论：整批墙钟、并发阶段均值与写入均值不能相加；吞吐分子须区分质量通过数。

读图讲解：先确定时间起止与分子分母；阶段条形不是可直接相加的总时间。

先区分保存路径改造与规划后端迁移

原始版：v1 同步基线

本报告的对照实现

规划：MotionGen
环境：8 环境生成配置
保存：主流程同步导出

压缩与写入完成后
生成控制流才能继续

已使用 GPU 与多环境
不是“整个系统完全串行”

改进版：v1 异步保存

主要改变保存侧

规划：保留 v1 后端
环境：保留 8 环境
保存：独立 CPU writer

共享内存移交数据
分片保存 → 排空 → 合并

生成—保存流水线重叠
另增加保存侧任务并行

cuRobo v2 适配版

主要改变规划侧

规划：MotionPlanner
环境：保留 8 环境
保存：沿用异步保存池

适配模型与目标表示
插值轨迹 → 关节重排 → FK

规划后端迁移与接口适配
不是新增 GPU 生产进程

当前依据：v1 同步基线、冻结异步保存实现及 v2 规划适配器。

支持结论：保存路径改造与规划后端迁移是两项变化；三版均保留 8 环境生成配置。

读图讲解：按保存侧与规划侧两条变化讲；原始基线本来就有 GPU 和 8 环境。

沿调用链说明具体改动

原始版：同步保存



同步导出占用生成控制流。

改进版：异步保存



Bridge.export → ExportPool.submit；提交后生成可继续，超预算仍会等待。

v2 版：规划适配 + 异步保存



规划路径在前端变化；episode 保存仍走共享内存与 writer 池。

当前依据：recorder、Bridge.export / ExportPool.submit 与 v2 plan_pose 的调用链。

支持结论：异步版转移保存工作；v2 版适配规划输入输出，示意图不表示实测时序。

读图讲解：顺着调用箭头说明谁持有数据、在哪里返回、何时文件才就绪。

代码差别在导出路径和扩容决策，共用 cuRobo v1 生成任务

同步 · workers=0	固定上限 · 4 / 8	auto
保留原 recorder 导出	ExportPool._scale_workers	ResourceGovernor.decide
episode 完成 ↓ 主进程压缩 + 写 HDF5 ↓ 继续生成	pending 有积压 ↓ desired = min(pending, cap) ↓ 按需启动 CPU writer	pending 有积压 ↓ CPU / 内存 / I/O 等准入 ↓ 通过：增加 1 个 writer
主流程承担保存等待	本轮两种上限均启动 4 个	至少间隔 1 秒；不主动缩容

当前依据：v1 实验冻结源码；同步、固定上限与 auto 共用生成任务。

支持结论：实现差别在保存路径与扩容策略；没有单项优化的消融结果。

读图讲解：先比较同步与异步，再解释固定上限与 auto 的扩容差别。

模块拆分：原 handler 的保存职责转入桥接层与进程池

原始：生成进程内

RecorderManager
录制 / export_episodes
↓
EpisodeData.pre_export
列表统一 torch.stack
↓
HDF5DatasetFileHandler
write_episode + flush

改进：生成主进程

RecorderManager（保留）
↓ 实例方法转接
Bridge + MetadataHandler
导出路由 / 元数据
↓
ExportPool
预算 / 任务 / 内存 / 回执
最终排空与合并

改进：CPU 子进程

spawn → writer
↓ 消费 jobs 描述信息
write_job
映射 CPU 共享内存
gzip + 独立 HDF5 分片
↓
results 返回 saved / failed
不初始化仿真与 CUDA

当前依据：原 RecorderManager / EpisodeData / HDF5 handler 与冻结异步模块。

支持结论：保存职责拆成桥接、任务管理、子进程写入与合并；上游文件未永久改写。

读图讲解：先找生成主进程与 CPU writer 边界，再沿数据与回执两个方向讲。

四个接入时刻：安装异步保存，不永久改写上游源码

创建环境前后

generate.build_source
按固定锚点改入口文本

创建前 configure(env_cfg)
替换类方法 add_to_episodes
设置 MetadataHandler 类型

创建后 attach(recorder, settings)
MethodType 绑定实例 export

生成运行中

compat.install_generation_compat
在 env_loop 中插入 poll

外层循环处理回执
等待动作时也处理回执

Bridge.export → pool.submit
主流程仍检查预算和打包
提交返回不等于保存完成

结束或异常时

达到生成目标 → finish
检查成功提交数
排空 → join → merge
关闭环境 → output_ready

异常 / finally / atexit
调用 abort 清理自有资源
锚点变化需重新核对适配

当前依据：generate.build_source、async_export.configure / attach 与 compat 的 poll 注入。

支持结论：启动注入、类方法替换和实例绑定是不同层次；异步 flag 控制接入路径。

读图讲解：按环境创建前、创建后、运行中、结束时四个时刻定位新增调用。

数据所有权：环境切片、源帧与共享内存分别管理

原始数据路径

录制：整批 value.clone()
每环境保留切片 clone=False
切片可继续持有整批存储

导出：pre_export
各字段列表 torch.stack
.cpu().numpy() → HDF5
统一堆叠仍在生成进程

改进 CPU 帧路径

录制：每环境独立 clone
导出：layout 计算偏移与形状
分配有界共享内存

逐帧 np.copyto
已复制源帧置 None / 清空
尝试归还分配器空闲页
不调用原统一 pre_export

必须区分的边界

GPU 状态列表仍先 stack
再转 CPU、复制到共享内存

源帧复制后可释放
共享内存等 saved 后释放

不是零拷贝 / CUDA IPC
没有独立峰值内存消融

当前依据：原 pre_export 的 torch.stack 与冻结 owned_recording_method / layout / pack。

支持结论：CPU 按环境持有且逐帧打包释放；仍有复制，没有独立内存收益消融。

读图讲解：区分源帧释放与共享内存释放；GPU 状态仍先堆叠再复制到 CPU。

完成语义变化：提交、分片保存和最终就绪分成三步

提交与保存

submit → entries=submitted
任务加入 pending 与 jobs

writer：started → write_job
写临时组 → 移动 → flush
返回 saved

poll 才释放内存并更新计数

最终文件

finish 等待 pending 清空
发送退出哨兵并 join

_merge 排序复制分片
检查属性 / shape / dtype
.partial → os.link 最终路径

env.close 后记录 output_ready

失败与限制

failed / 进程退出 / 超时
由父进程 poll 报错
abort 终止 writer 并回收内存

合并检查不等于逐值校验
没有多输出事务回滚保证
没有 fsync 持久化承诺
末态质量仍需独立验收

当前依据：ExportPool.submit / poll / finish / _merge / abort 与 writer 回执协议。

支持结论：submitted、saved 与最终就绪分离；异常需传播并清理，不能把提交当完成。

读图讲解：先沿成功路径到最终文件，再看失败、超时及进程退出如何回到父流程。

流程分段：并行化发生在哪里，哪里仍需等待

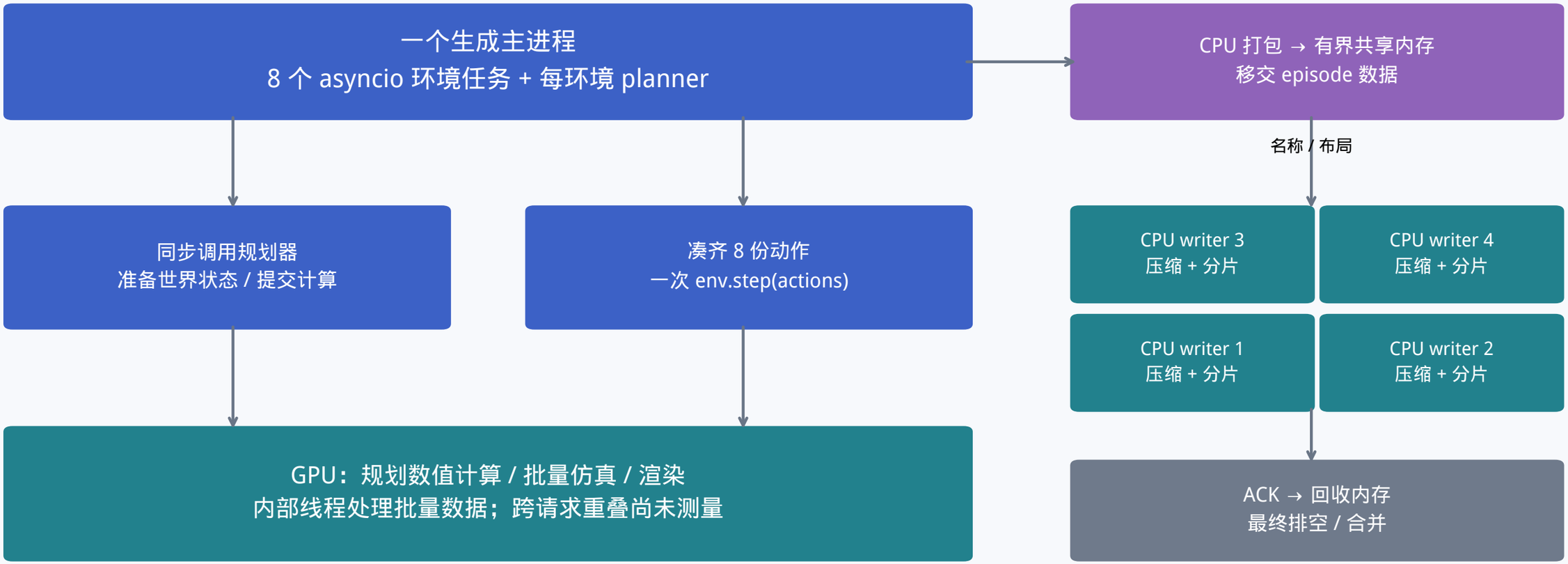
阶段	执行方式	同步点与边界
任务调度	asyncio 协程并发	共享主线程；等待未完成时挂起
规划计算	GPU 算子内部数据并行	主机逐环境调用；跨请求重叠未测
仿真步进	动作汇合后批量 env.step	等待整批动作；不是每环境一个进程
录制 / 内存移交	主流程 clone、pack、submit	CPU 帧所有权调整；仍有打包与预算等待
压缩 / 写入	spawn CPU 进程处理不同 episode	独立 HDF5 分片；共享 CPU 与磁盘资源
回执 / 收尾	主流程 poll、finish、merge	回执后回收内存；排空后合并最终输出

当前依据：环境主循环、规划调用、录制包装与冻结保存池源码。

支持结论：各阶段采用不同并发机制；规划跨请求重叠与渲染并发尚无设备追踪证明。

读图讲解：逐行说明执行单位和等待点，避免把协程、GPU 和 writer 数量混为一谈。

执行层次：协程并发、GPU 数据并行、CPU 多进程



架构示意：8 个环境任务共享生成主进程；本轮异步组实际均为 4 个 CPU writer。

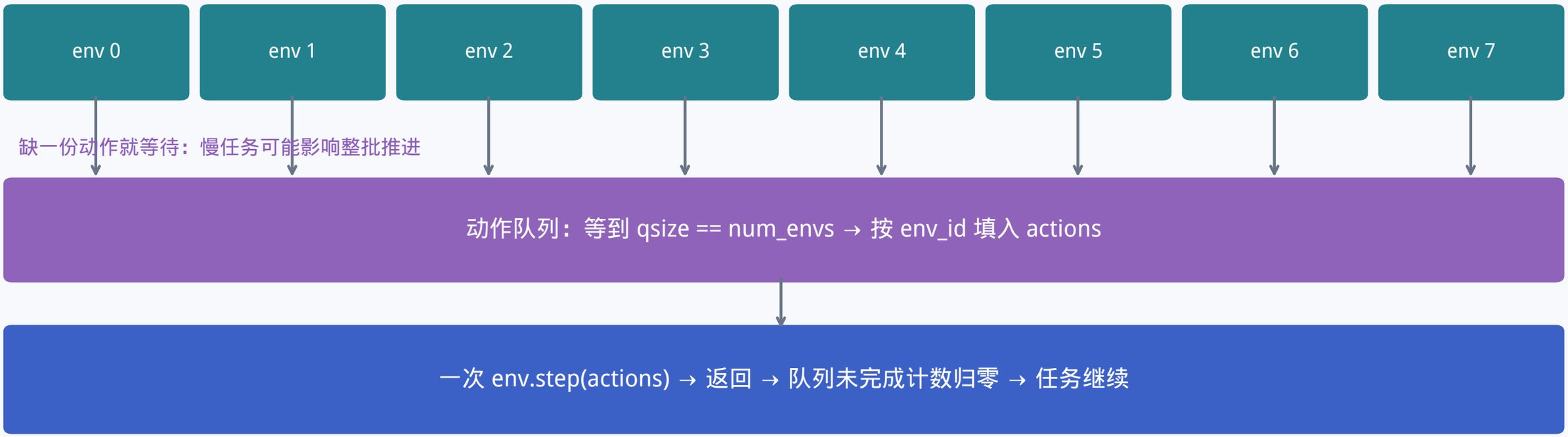
当前依据：主循环源码与保存池配置；本轮异步组实际启动 4 个 CPU writer。

支持结论：环境任务、GPU 计算与 CPU 保存分属不同层次；未测跨请求 GPU 重叠。

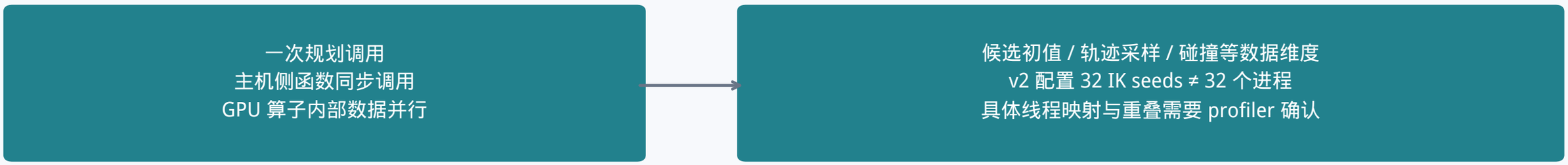
读图讲解：左侧是生成与 GPU 计算，右侧是 CPU 保存进程；共享内存负责数据移交。

环境在动作屏障汇合；GPU 算子内部执行数据并行

主进程：环境提交动作后等待 queue.join()；等待未完成时挂起协程



下方是规划计算原理示意，与上方动作屏障不是同一条执行时间线。

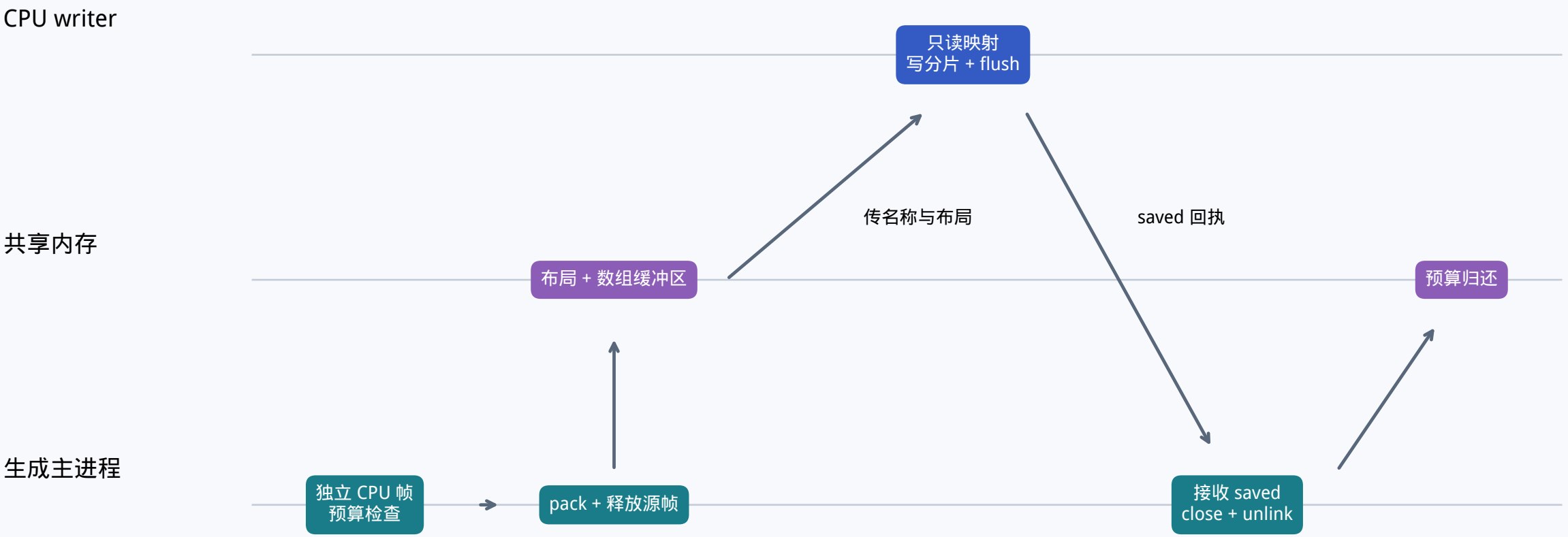


当前依据：动作队列汇合、env.step、task_done 与规划调用的代码路径。

支持结论：存在主机侧批次等待；队列完成记账不等于 CUDA 设备同步。

读图讲解：先讲动作凑齐才能步进，再讲 GPU 算子内部并行；两者不是同一时间线。

数据所有权与回收：必须等 writer 确认后才能释放



示意图，无时间比例；队列预算不足会等待回执，writer 写入可与下一条生成重叠。

当前依据：冻结源码中的共享内存打包、saved 回执和释放流程。

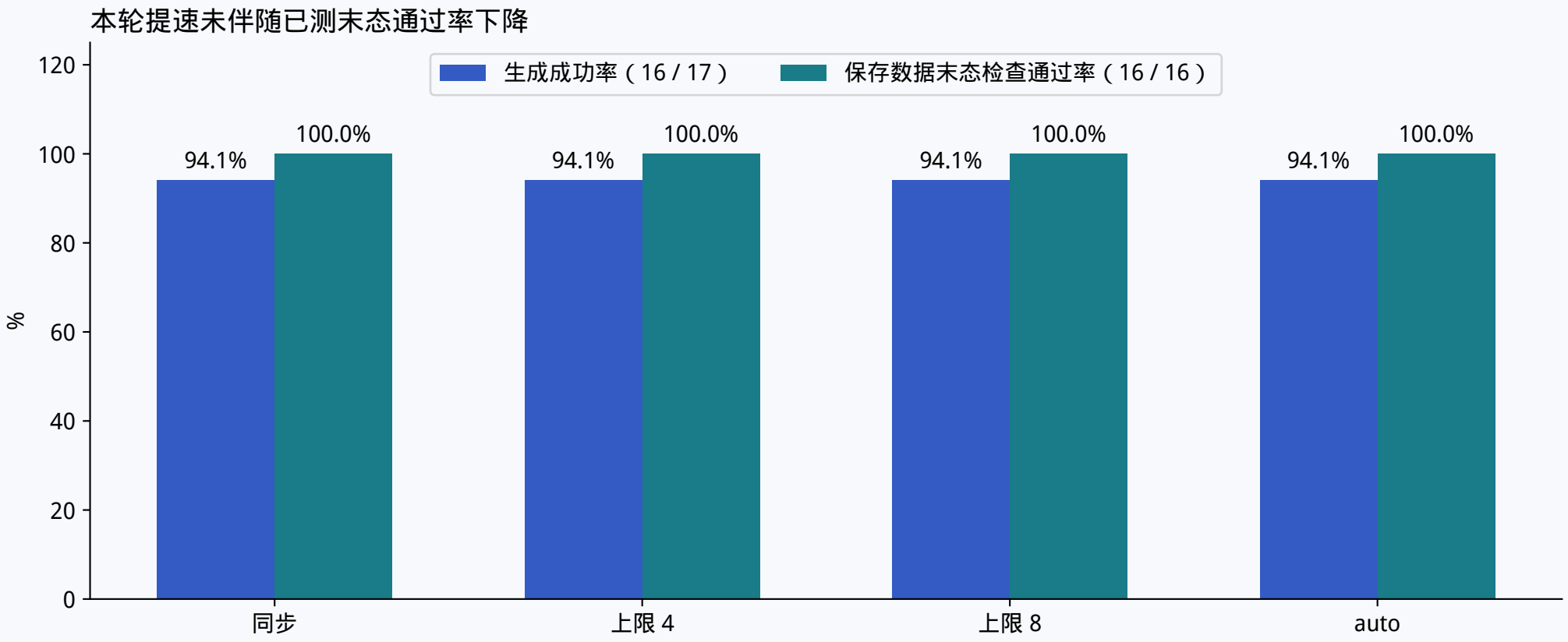
支持结论：回执后释放数据并归还预算；未单独测量该机制的提速贡献。

读图讲解：从源帧到共享内存再到分片；只有收到保存回执后才能释放内存预算。

当前实验：v1，8 环境、seed 101，每组最终保存 16 条，单轮比较。

支持结论：异步组吞吐提高约 55%–66%；不能据单轮结果确定稳定排名。

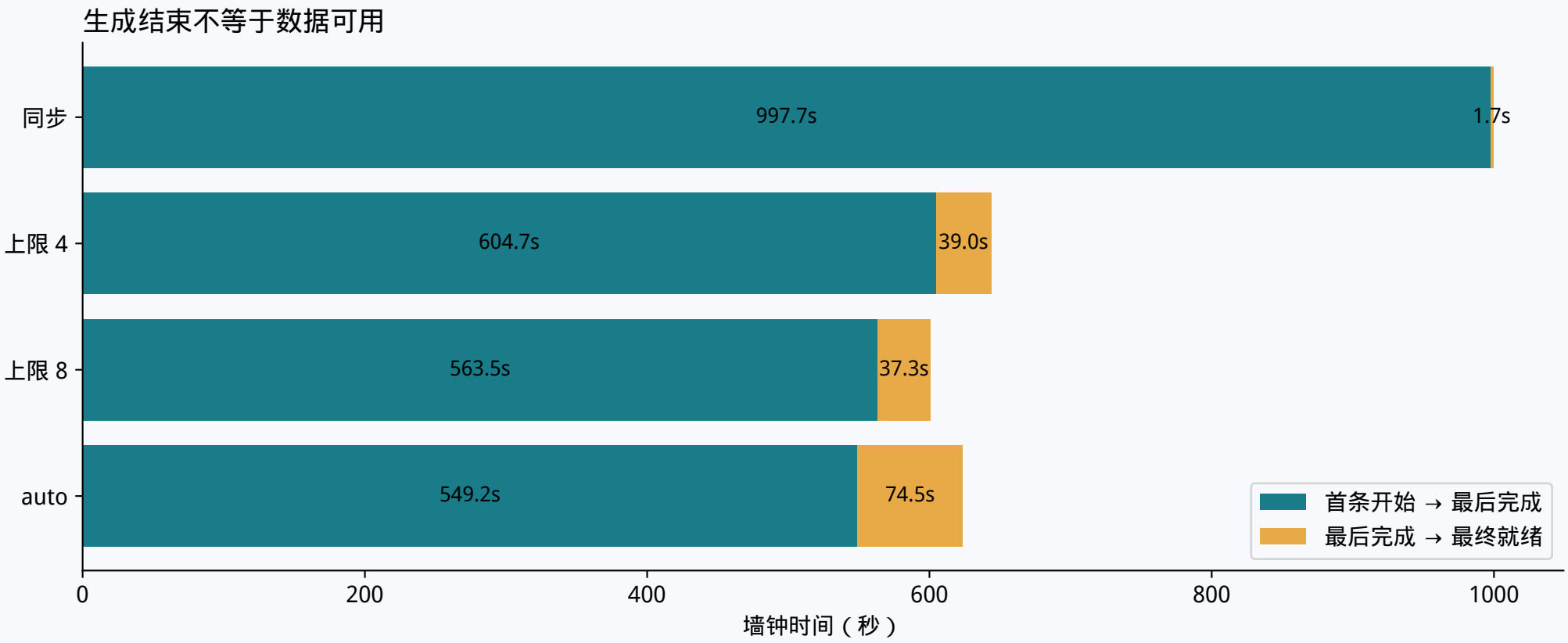
读图讲解：先看最终就绪耗时下降，再读吞吐增加；cap8 与 cap4 实际都为 4 writer。



当前实验：各组生成成功 16/17，保存数据末态检查通过 16/16。

支持结论：本轮已测末态通过率未下降；不代表全部质量指标或长期稳定性。

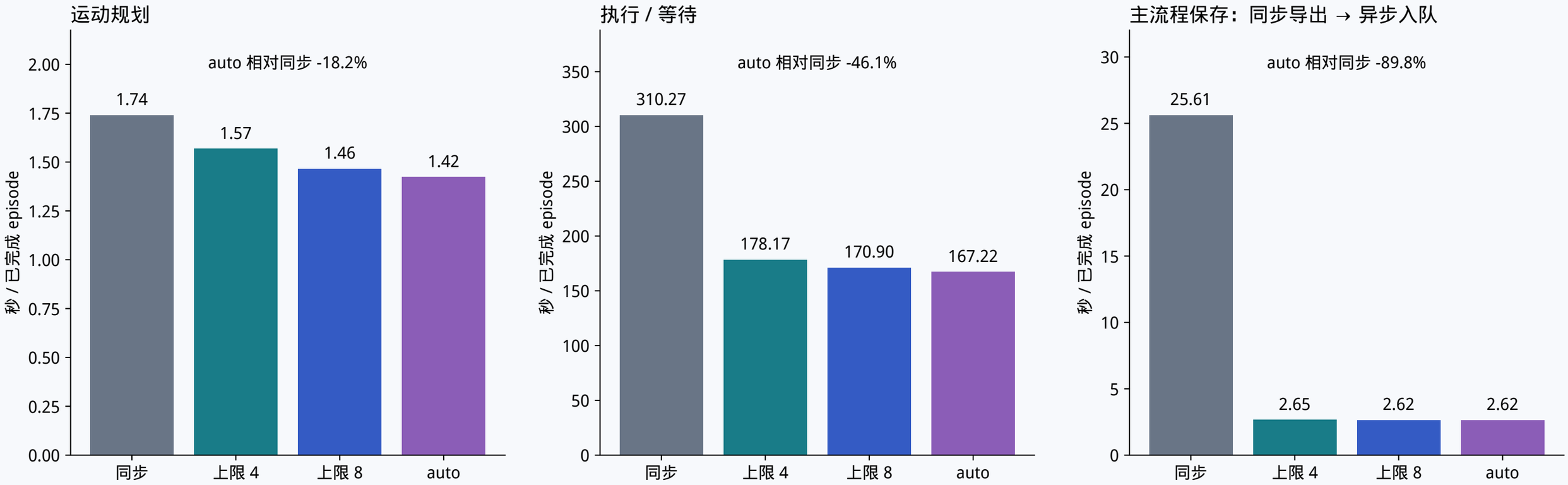
读图讲解：分开读生成成功率与保存数据末态通过率，说明速度收益对应的质量范围。



当前实验：将首条开始至最终就绪拆分为生成区间与收尾区间。

支持结论：最后一条生成完成后仍需收尾；生成结束不能代替文件就绪。

读图讲解：读整条总长，再比较收尾；auto 生成区间较短并不代表最终就绪最快。



各组 17 个完成样本（含 1 个失败）；右图两端工作不同，反映主流程负担而非后台写入加速。

当前实验：v1 四组各 17 个完成 episode 的规划、执行 / 等待及导出计时。

支持结论：主流程负担降低；同步导出与异步入队工作范围不同，不是写盘速度比。

读图讲解：执行 / 等待下降最明显；2.6 秒是移交负担，不能称作单条保存完成时间。

执行 / 等待 = MultiWaypoint.execute 整段调用的墙钟时间

计时包含的顺序

time.monotonic() 开始
↓
目标 / 噪声 → 动作转换
↓
动作 put → await queue.join
↓
成功条件检查 → bool
↓
正常返回后累计耗时

queue.join 期间可能发生

等待其余环境提交动作
整批 env.step(actions)
仿真 / 观测 / 录制等路径
队列完成后的协程恢复延迟

其他环境同步规划或导出
也可能占用共同事件循环线程

旧日志无法直接拆出的量

纯 physics kernel 时间
纯渲染与相机处理时间
动作屏障占比
主机与设备同步占比

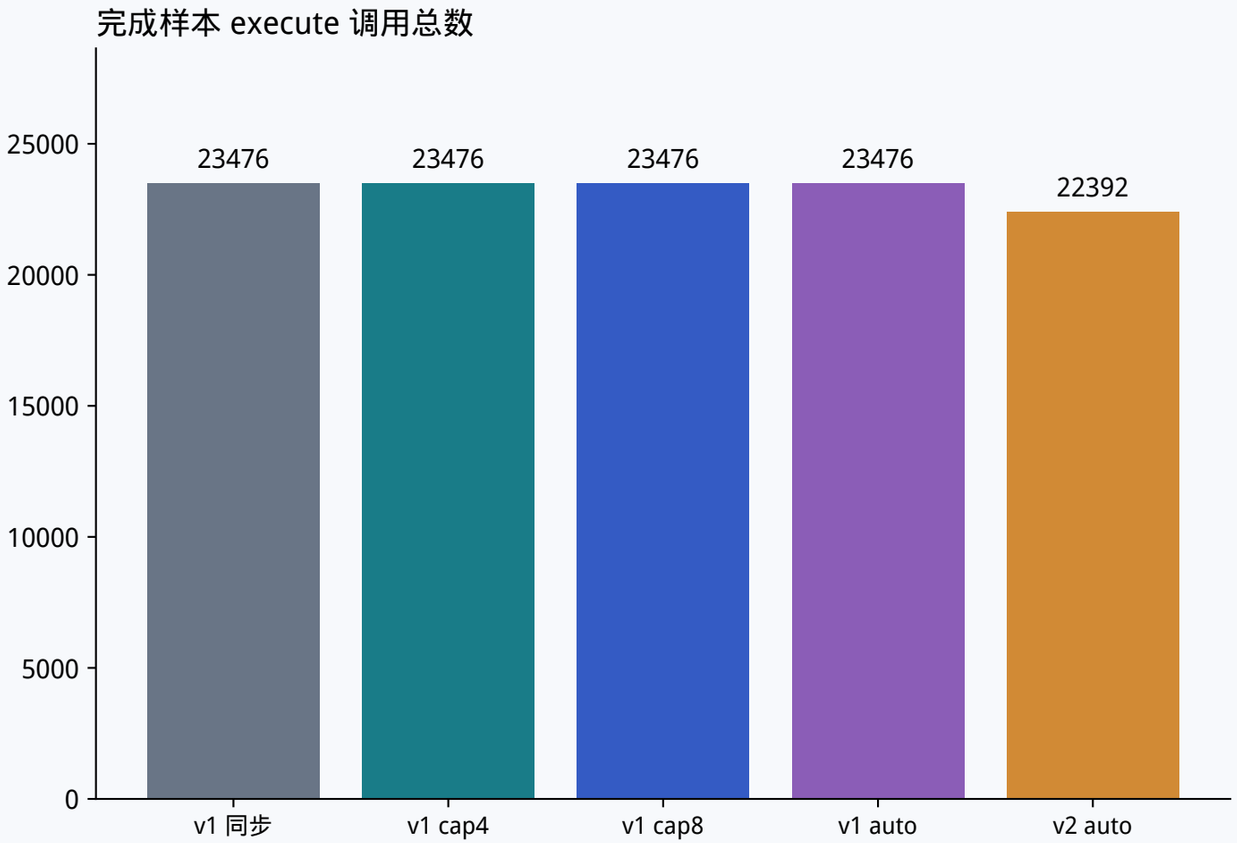
同一等待可被多个环境累计
不能按 GPU 忙碌时间解释

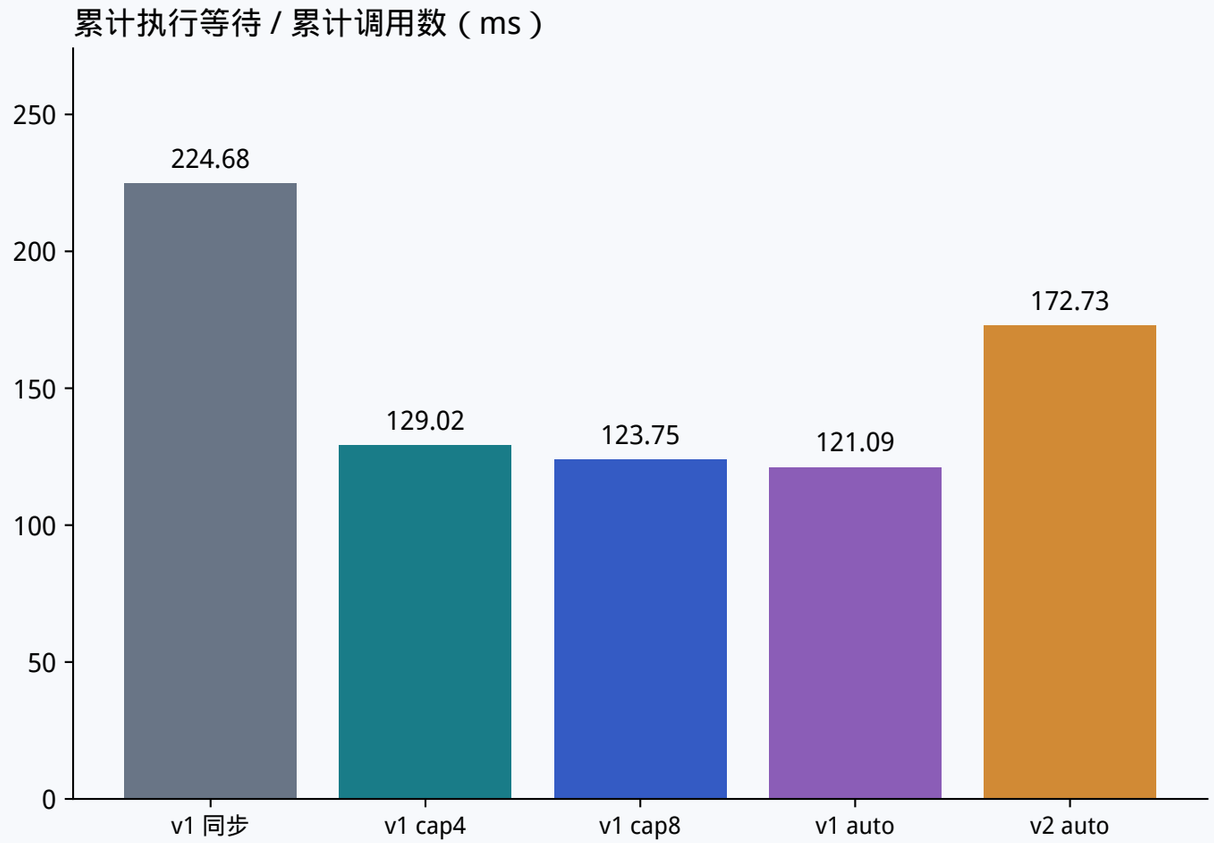
当前依据：冻结 telemetry.py 的 tracked_execute 与当前 waypoint / env_loop 源码。

支持结论：字段累计整个 execute 墙钟，含共享主线程和批次等待，不是纯仿真耗时。

读图讲解：先画出计时包围范围；其他环境的阻塞可能重复体现在本环境等待中。

v1 调用数逐样本一致：改善来自等待延迟下降，而非动作数减少



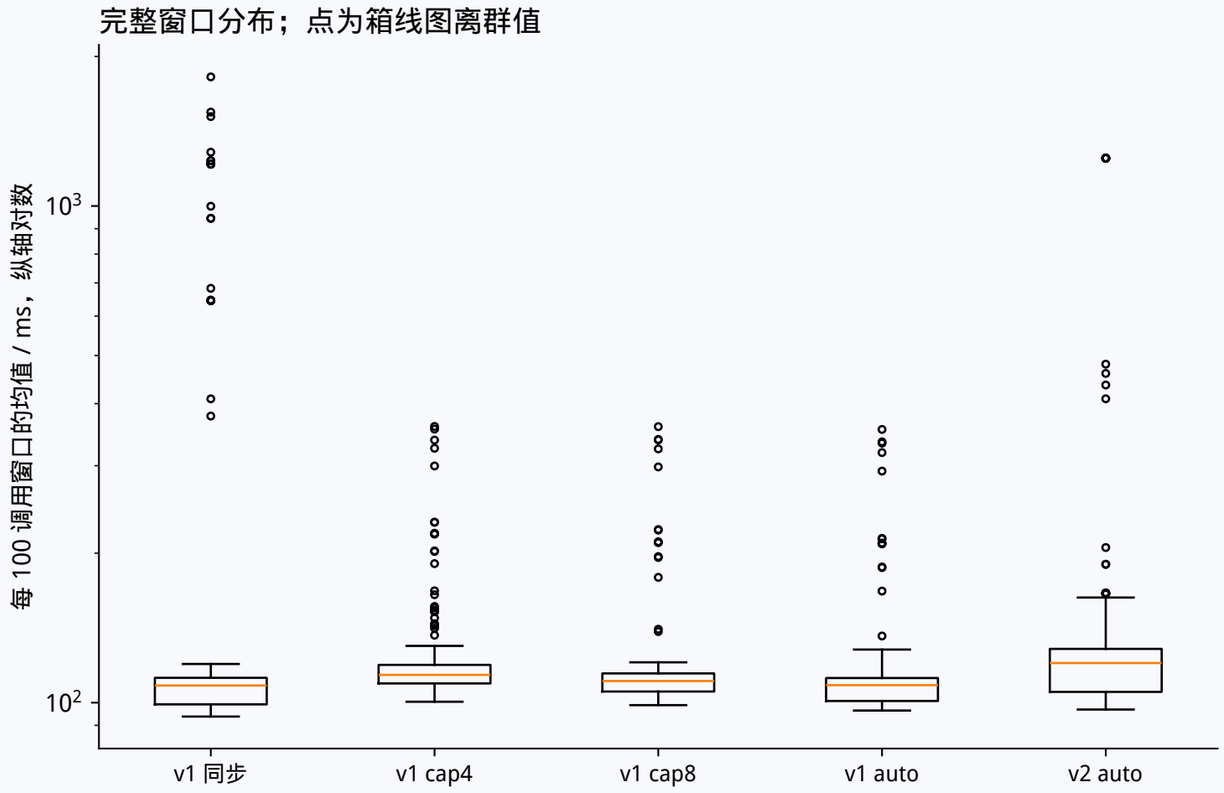


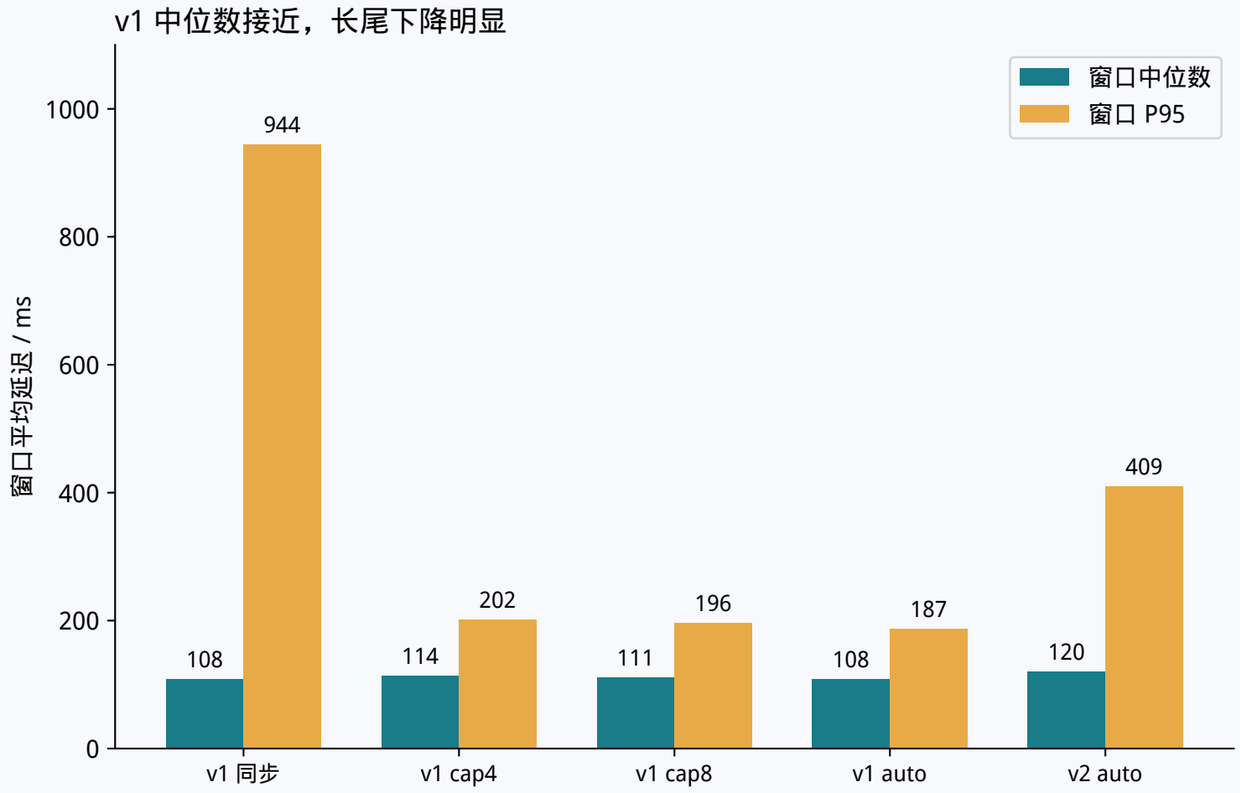
当前复算：只统计完成 episode；v1 各 17 条，v2 为 16 条，取消样本排除。

支持结论：v1 四组调用数相同，延迟下降不是少执行动作；v2 仍受跨日比较限制。

读图讲解：将总耗时分为调用次数与平均延迟；调用次数不是全局物理子步数。

区分典型速度与长时间停顿：窗口 P95 不是单次调用 P95

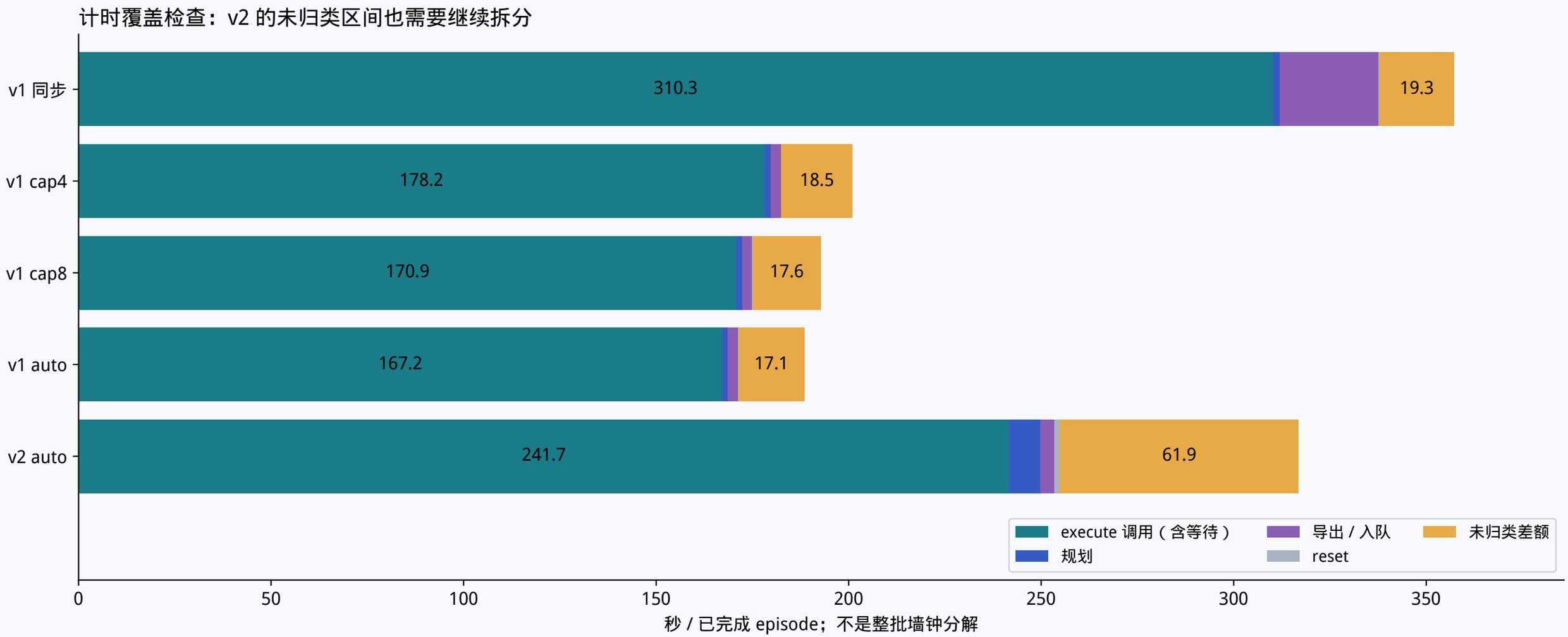




当前复算：v1 每组 225 个完整 100 调用窗口，v2 为 216 个，末尾片段不计分位数。

支持结论：v1 典型窗口接近、长尾明显下降；这是窗口均值分布，不是单次延迟 P95。

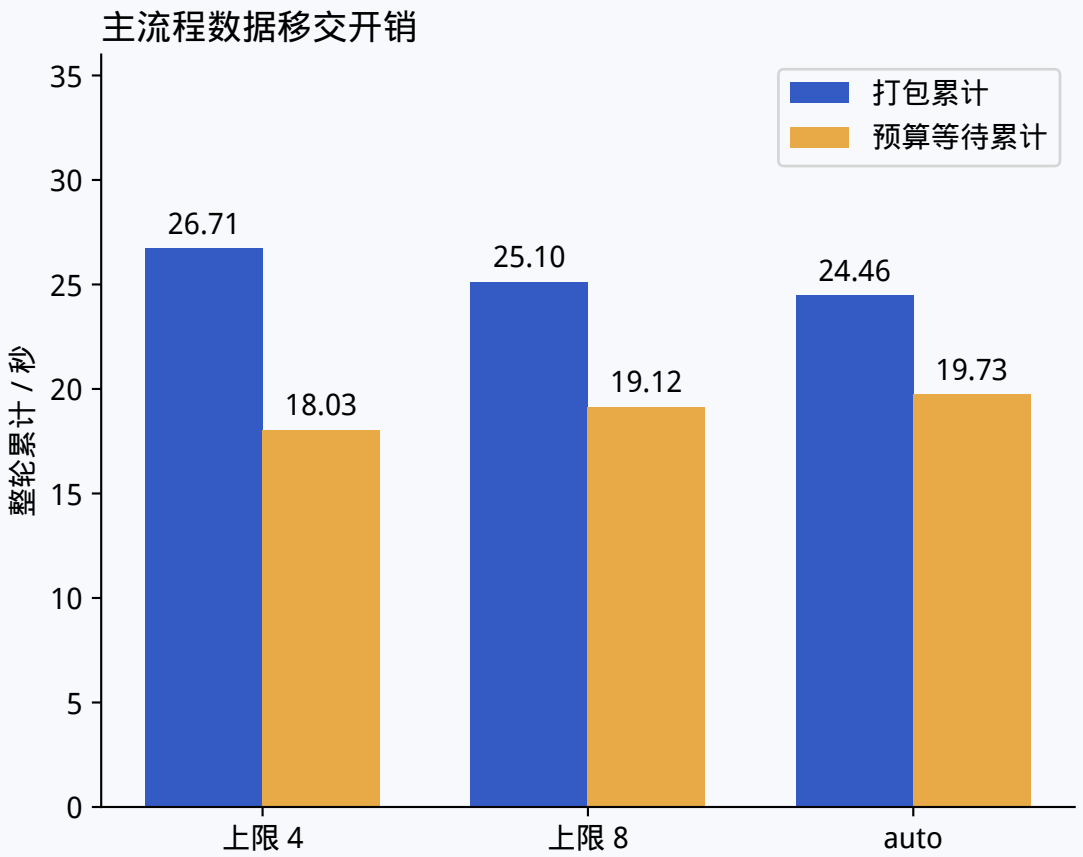
读图讲解：先看约 108 ms 的中位数，再看同步与异步的长尾差距。

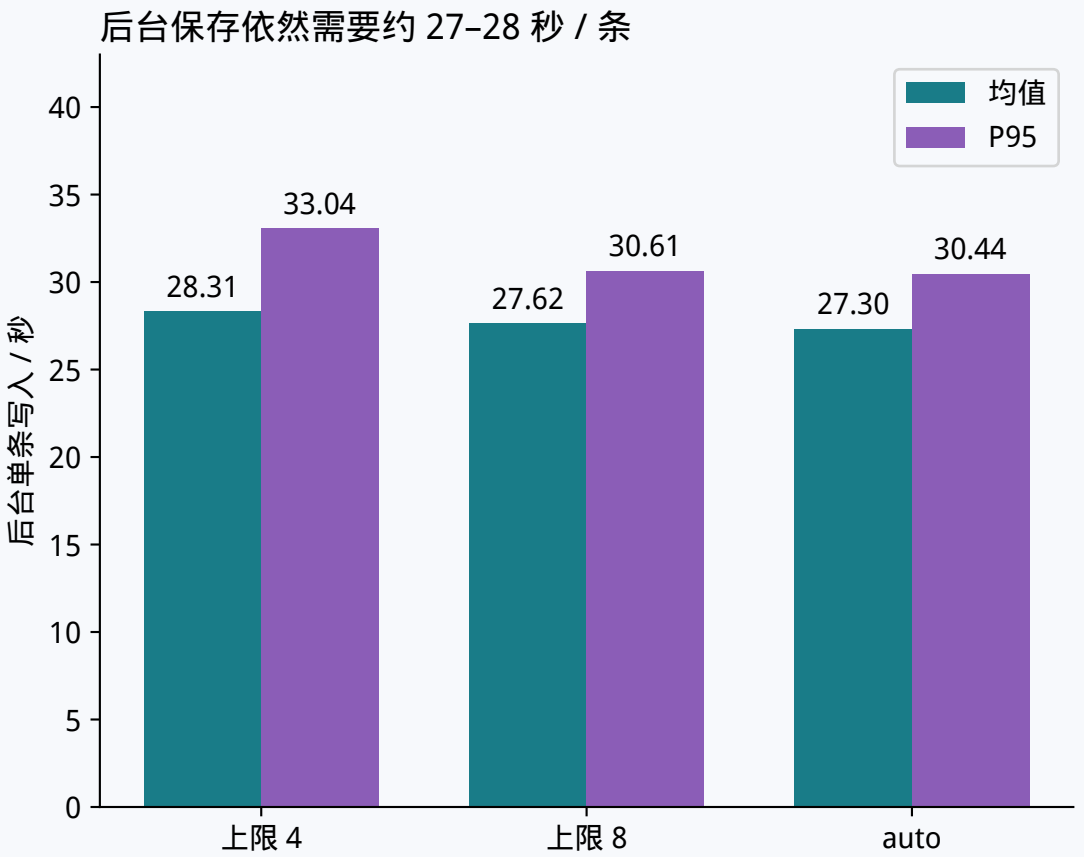


当前复算：episode 总时长减已记录阶段，差额非负；阶段与原始数据未改写。

支持结论：v2 另有约 61.86 秒 / episode 未归类；不能直接命名为 CPU 或渲染耗时。

读图讲解：计时覆盖不等于资源占用；执行等待比例高不代表 GPU 一直在忙。



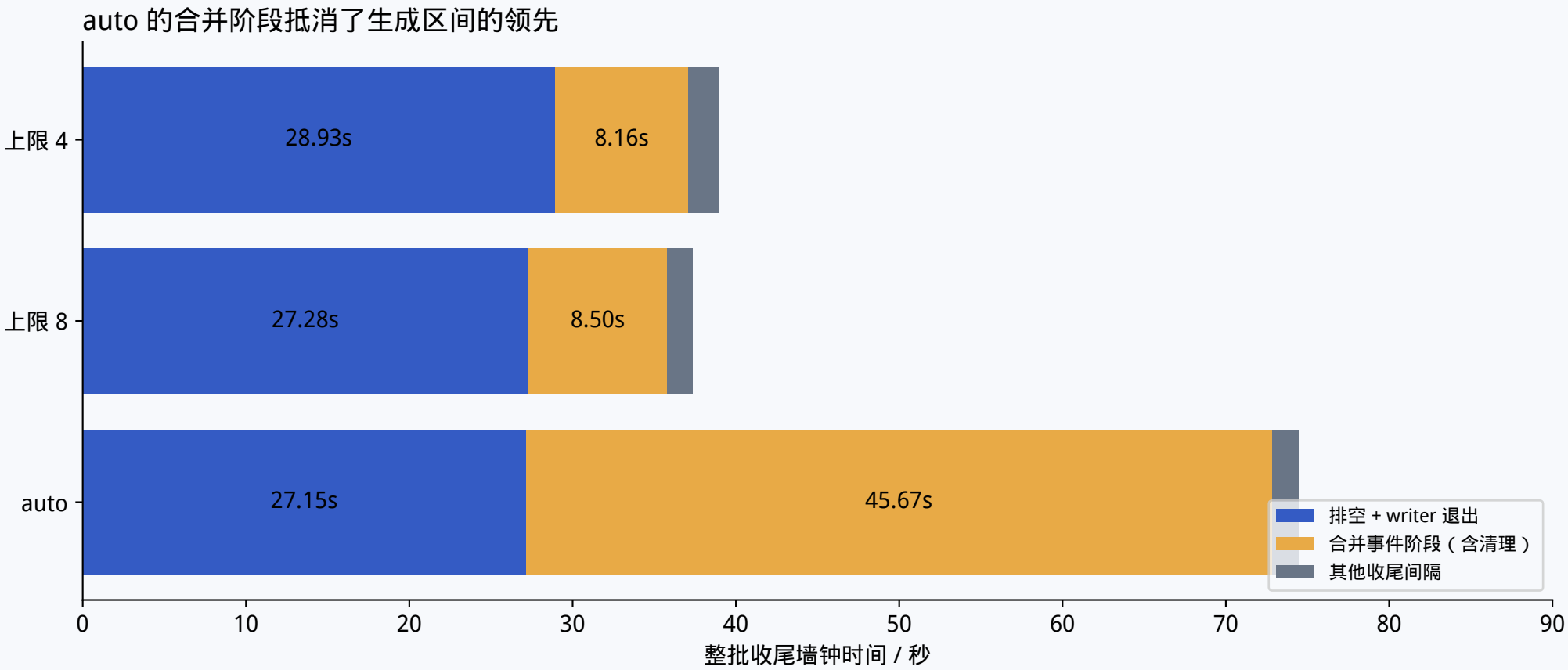


右图每组 16 个保存事件；后台并发耗时不能与左图相加为墙钟时间。

当前实验：异步三组打包与预算等待累计值，以及每组 16 次后台保存计时。

支持结论：数据移交仍有开销，后台每条保存约 27–28 秒；并发耗时不可直接相加。

读图讲解：左侧是主流程累计开销，右侧是后台单条计时；后台工作仍然存在。

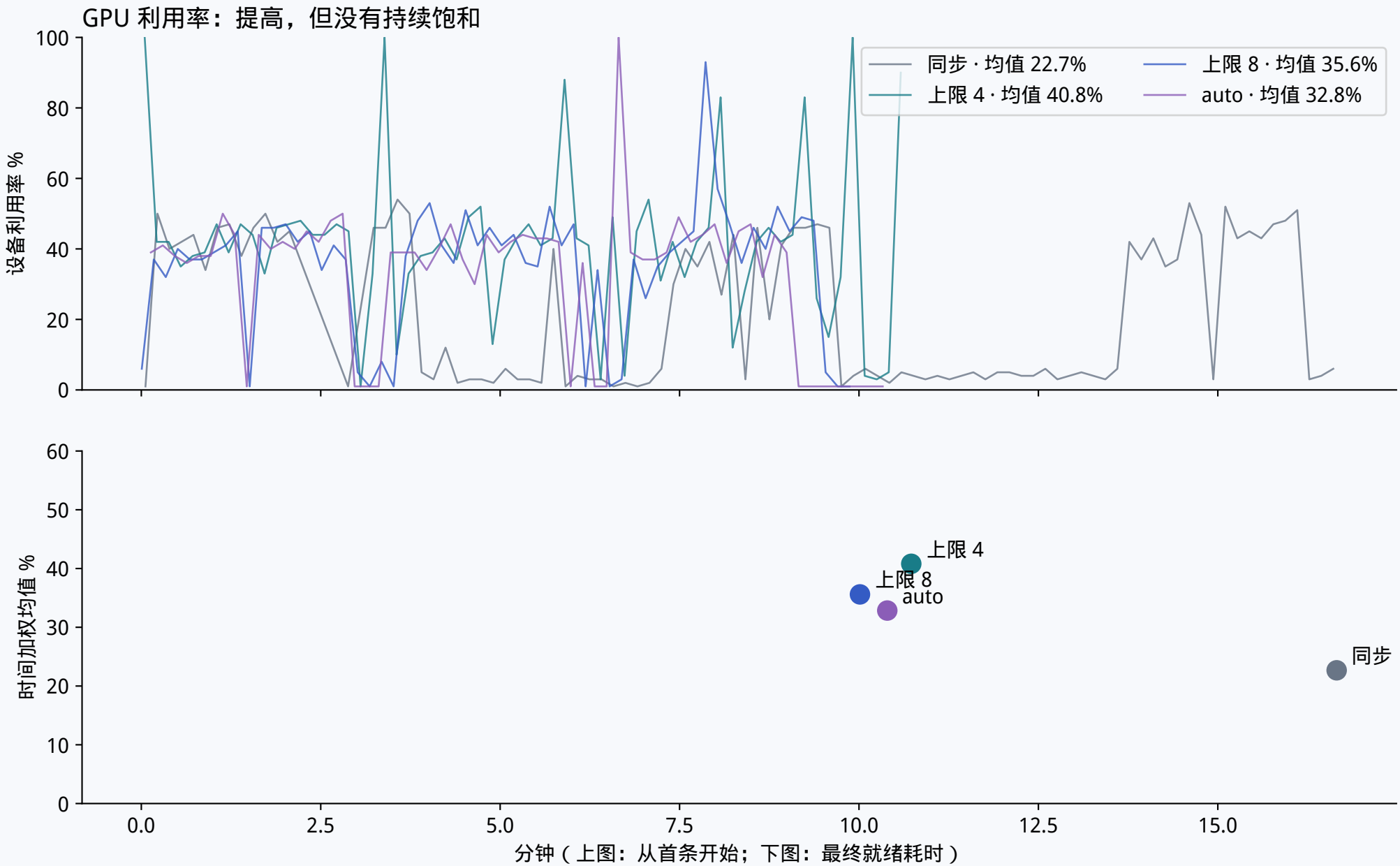


区间：最后 episode 完成 → output_ready。其他间隔为差值，并非独立埋点；同步收尾仅 1.68 秒。

当前实验：异步三组从最后 episode 完成到 output_ready 的收尾计时。

支持结论：本轮 auto 的合并事件阶段较长；单次结果不能证明 auto 必然更慢。

读图讲解：比较排空与合并；auto 的收尾长发生在本轮，不能推断长期排名。



均值采用首条开始至 output_ready 内有效采样的左保持时间加权；设备级指标包含其他进程。

当前实验：v1 四组运行期间的设备级 GPU 利用率采样。

支持结论：异步组平均利用率较高；不能证明 kernel 重叠或 GPU 算法提速。

读图讲解：把利用率当辅助证据，与完整吞吐对照；利用率高不自动等于更快。

GPU 工作提交受主线程闭环约束：保存并行没有拆开整条路径

等齐动作 → 批量 step

各环境准备动作
↓ put / queue.join
等待队列达到 num_envs
↓
env.step(actions)
↓ task_done
各协程继续下一步

主线程仍执行什么

同步规划 plan_single
成功标量 item / bool
观测 GPU → CPU
录制与共享内存 pack
预算不足时等待回执

这些路径可能推迟下一次提交

对应源码位置

env_loop：250–275 行
waypoint：333–336 行
planner：1312–1314 行
cameras：54–55 行
export_pool：296–314 行

源码跳转：gpu-analysis.html

历史依据：8 环境动作队列与冻结导出源码；新增 4 环境实测在后续页面单列。

支持结论：异步 writer 不会自动消除主线程提交间隙；屏障等待也可能伴随 GPU 规划。

读图讲解：沿等齐动作、step、观测和下一动作读闭环，再指出保存只移走一部分工作。

历史相机路径与新增配对实测：重复计算需要按状态版本复用

每个 RGB term

update_mounted_camera_poses
循环更新三个相机
↓
读取一组相机 RGB
↓ .to(cpu, copy=True)
CPU 图像副本
↓ observation term.clone()

一次 step 的条件计数

三路 RGB term 启用
且 recorder active
且加载当前 RL step

录制前 compute
重置 / 事件后 compute
→ 六次批量 RGB 回传调用
→ 十八次 set_world_poses

优化须保留语义

相机刷新可评估集中处理
CPU 回传可评估有效缓存

reset / 事件后可能已变状态
不能直接删第二次 compute

调用数不等于渲染次数
需要实测计数与设备追踪

历史依据：冻结相机配置与当前 RL step；新增 4 环境插桩已核对加载文件与调用数。

支持结论：新配对确认六批 RGB 副本与约三次渲染；历史 8 环境依赖身份仍需区分。

读图讲解：先讲 CPU 副本的内存目的，再讲重复计算；不能删除重置后的观测更新。

先定位设备空隙的来源，再评估批量大小和并行策略

优先验证

1. 加载路径 / hash / 调用数
2. 观测与 RGB 复制时间线
3. 每环境动作就绪时刻
4. bool / item 与规划区间
5. pack 与预算等待

CPU 区间需关联 CUDA 时间线

分开做消融

相机刷新与缓存复用
成功判定批量计算 / 读回
规划与 FK 批处理
共享内存打包调度

单项改变，检查输出一致性
诊断追踪与吞吐运行分开

已有证据与边界

v1 调用数一致，长尾下降
尚无各段 GPU 耗时占比

利用率不是 SM occupancy
整批均值还包含收尾阶段

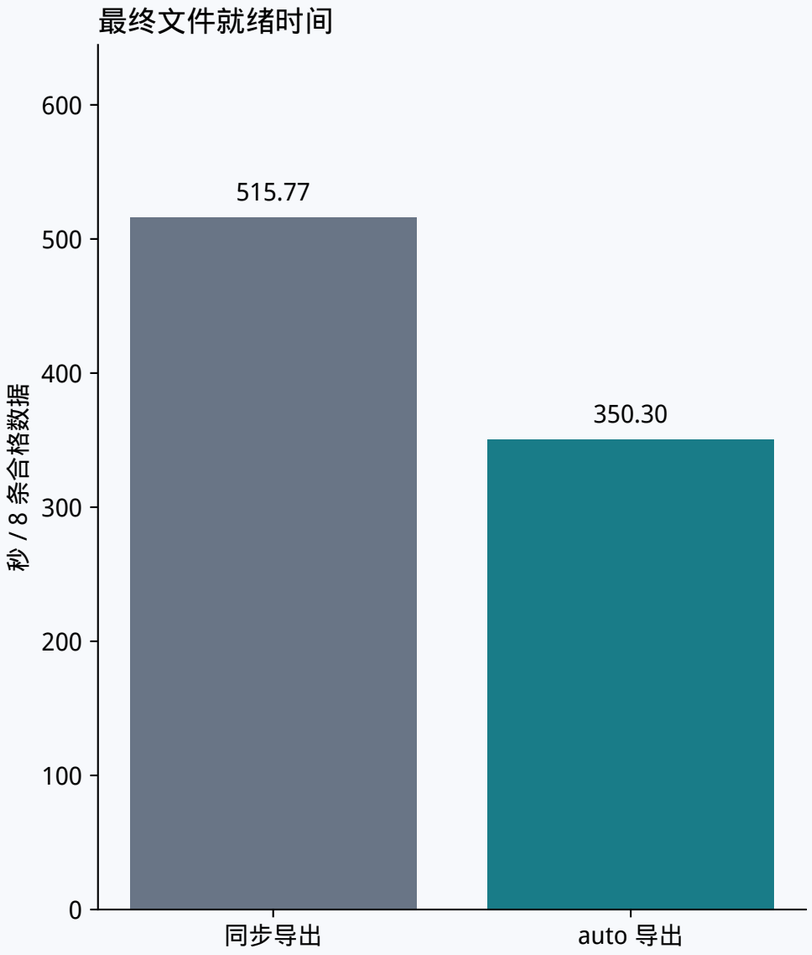
最终评价：合格产出 / 时间
不以利用率越高越好为目标

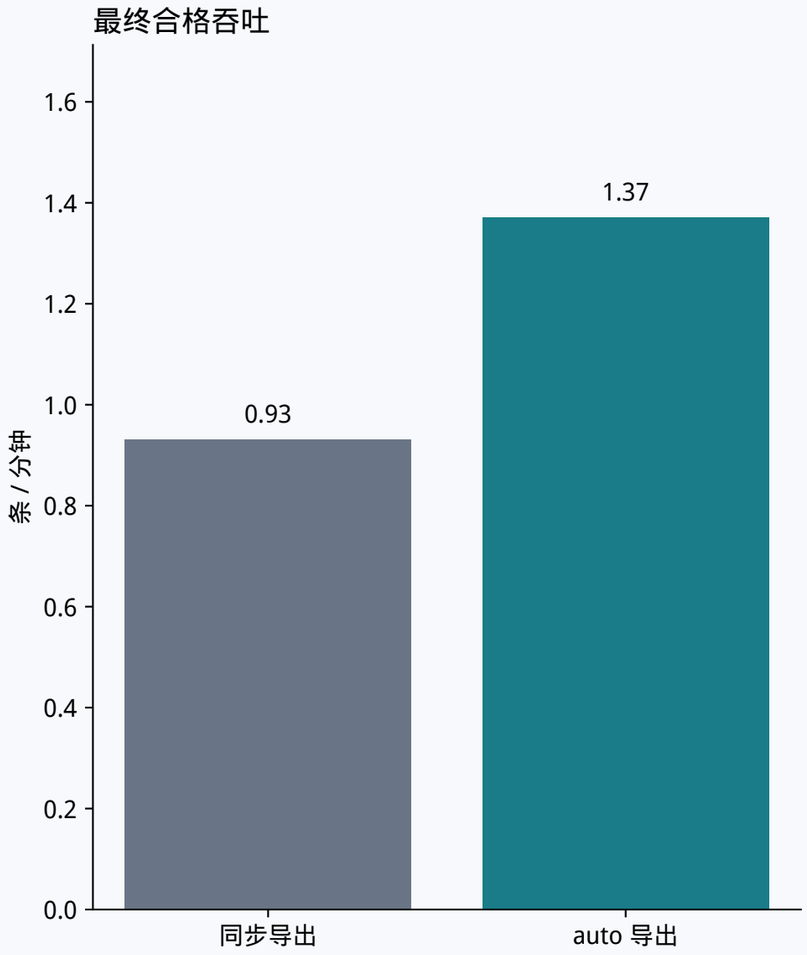
证据边界：历史 8 环境没有内部分段；后续新增 4 环境主机计时，仍无 kernel 追踪。

支持结论：主机分段已可量化，设备重叠与缓存优化收益仍需独立消融验证。

读图讲解：先验证相机路径与动作就绪，再做单项消融；以合格吞吐作为优化判据。

新增独立配对：主机分段计时补齐内部归因，历史 8 环境继续单列





共同条件与检查

同一 RTX 5090 / cuRobo v1
4 环境，seed 101
各 8 条成功目标
60 个最终保持步

两组严格终态 8/8 通过
结构检查通过
8 组动作序列精确匹配

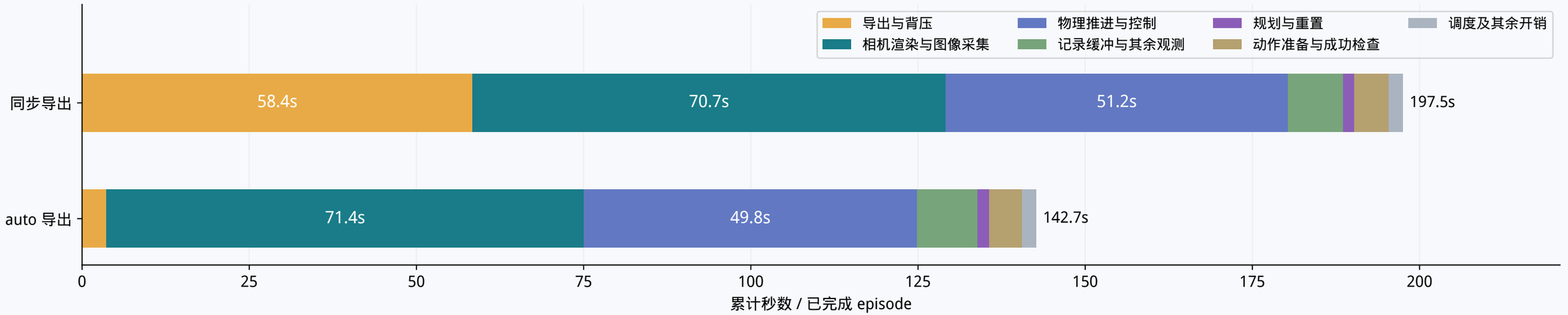
完整文件吞吐提高 47.2%
新缓存方案尚未实施

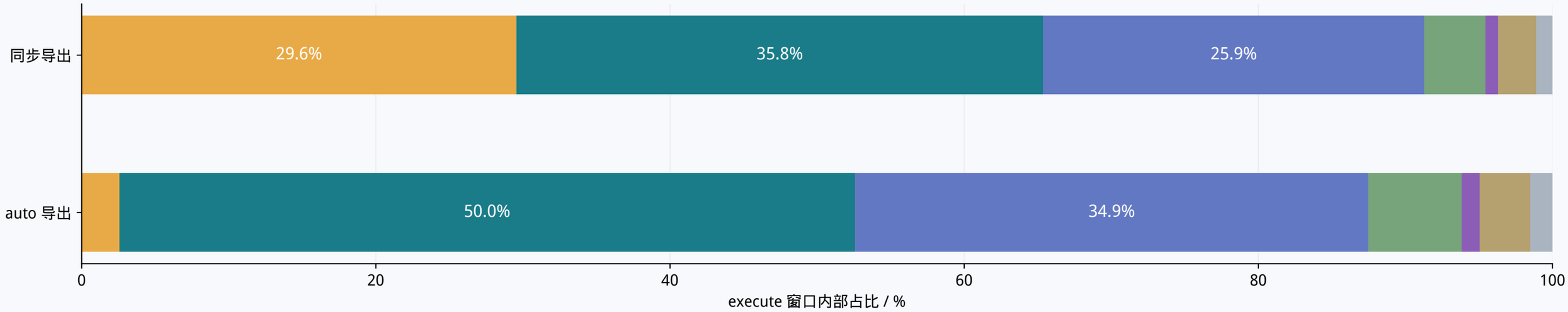
当前实验：2026-10-15，v1 同步 / auto 各 4 环境、8 条成功目标、seed 101。

支持结论：两组 8/8 严格终态通过、动作序列精确匹配；最终文件吞吐提升 47.2%。

读图讲解：这是独立的 4 环境配对，不能与前面 8 环境历史实验混成同一组。

相机绝对耗时接近，占比升高来自导出阻塞减少后的分母变化





当前测量：同步作用域互斥计时；统计完成 episode 的 execute 窗口，取消样本排除。

支持结论：execute 减少 54.82 秒，其中导出相关减少 54.77 秒；相机绝对耗时接近。

读图讲解：主机 API 墙钟包括阻塞；并发窗口会重叠，百分比不是 GPU kernel 占比。

物理推进与控制的分工：主机调度、GPU 工作和阻塞分别理解

环节	在做什么	CPU / GPU 分工	auto execute 内部
物理推进	碰撞、接触、关节与刚体运动 推进机器人与方块的位置 / 速度	GPU PhysX 批量推进 4 环境 主线程调用含提交与等待	36.66s / 25.69%
控制与状态	动作处理、控制器命令 写入仿真与更新状态缓冲	CPU 调度 + GPU 张量运算 不代表全部在 CPU 上计算	13.15s / 9.21%
相机位姿	机器人 link 世界位姿 × 安装变换 设置三路相机的位置和方向	CPU / GPU 混合 线程 CPU / 调用墙钟约 99.3%	36.14s / 25.32%
相机实际渲染	从当前场景生成 RGB 物理状态决定本帧相机与物体位置	GPU RTX 渲染 主线程比值约 45.6%	28.97s / 20.30%
显式 RGB 副本	将图像从 GPU 复制到 CPU 交给 recorder 留存	GPU→CPU 传输 + 主机处理 完成后才能读取该副本	4.16s / 2.91%

当前依据：运行日志确认 cuda:0、PhysX 与 RTX；已记录主线程墙钟及线程 CPU 时间。

支持结论：GPU 批量物理与渲染已启用，控制、位姿设置和 CPU 图像副本仍有主机成本。

读图讲解：先说明每个环节做什么，再区分 CPU 调度、GPU 工作和主机等待。

批量环境内部并行；同帧阶段按状态依赖推进



环境维度：同一 GPU 批量推进多个环境，GPU 算子内部存在数据并行。

阶段维度：本帧相机读取依赖物理结果；CPU 消费依赖图像传输完成。

实际每 batch step：4 次物理推进、约 3 次相机渲染、6 批 RGB CPU 副本。

箭头只表示依赖；未采集 GPU 时间线，不能据此断言设备阶段完全串行或已经重叠。

当前依据：实际 step 调用顺序和计数；4 个环境共享整批动作屏障与一条主循环。

支持结论：环境内部可批量并行；同帧物理、相机刷新和采集有数据依赖，重叠未测。

读图讲解：此图是依赖关系示意，不是 GPU 时间线；计数不能证明 kernel 同时运行。

首选去重方案：保留两次观测，只复用同状态下的昂贵相机结果

当前正常路径

每步两次观测计算
每次读取三路 RGB

每个 RGB term 刷新全相机
6 次全相机刷新
→ 18 次相机位姿设置

6 批 RGB CPU 副本
实际渲染已约为 3 次

待验证的快路径

有效状态版本内
统一刷新三路相机

6 次全相机刷新 → 1 次
18 次位姿设置 → 3 次

同帧 CPU RGB 副本复用
6 批 → 3 批
固定索引和变换也复用

缓存失效与所有权

reset_epoch / state_revision
相机 pose_revision / frame_id

重置与事件后立即失效
保留第二次 compute/history
不接受过期帧或一帧延迟

每路只持有有界单帧
recorder 按环境持有不变

当前测量：正常 step 两次观测、六批 RGB 副本、六次全相机刷新；实际渲染约三次。

方案目标：按有效状态版本复用相机位姿与单帧 RGB；保留 reset / 事件后的更新。

读图讲解：6→1 与 6→3 是待验证的调用目标；GPU 渲染本来已有缓存，不能再算减半。

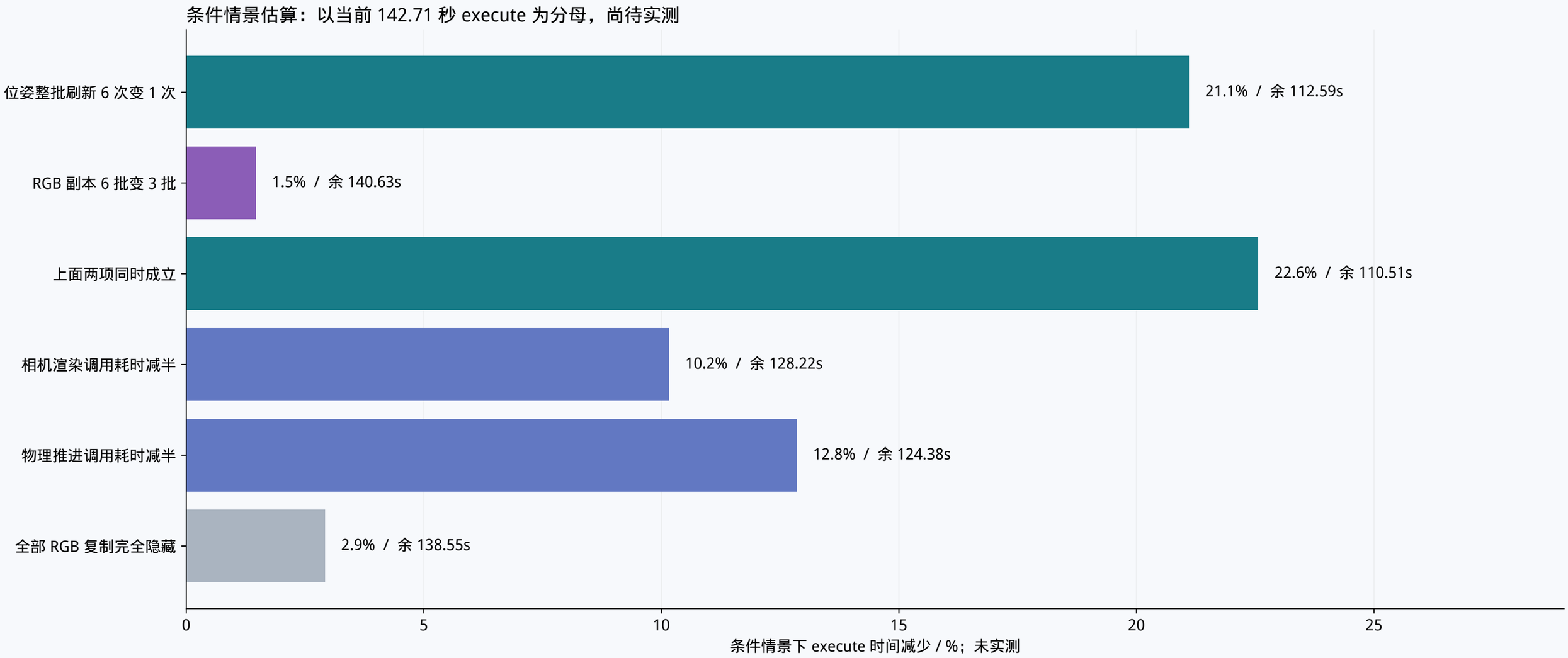
按实测占比选择优化，以最终合格吞吐评价

顺序	方案	实测依据	正确性与收益边界
P0	位姿刷新去重 复用固定索引 / 变换	25.32% 主线程占用明显	保持 reset / 事件 / 位姿失效语义
P1	同状态 RGB CPU 副本复用 保留完整观测计算	2.91% 显式复制 未含任意观测的全部成本	帧序、history、不可变性、内存有界
P2	设备时间线后定位 renderer 实际跨相机提交和状态同步	20.30% 渲染调用 不等于隔离 GPU kernel 时间	本版本 TiledCamera 已是别名 不能只换类名期待加速
P3	定位物理提交 / 求解 / 等待 批量成功判定与小张量操作	物理推进 25.69% 控制 / 更新另 9.21%	先保持 dt、子步、接触与成功阈值
后置	有界 pinned 缓冲 + 异步 D2H 打包与规划/FK 分别消融	复制 2.91% auto 导出相关 2.55%	事件完成后 CPU 才读；禁止提前复用 不盲目增加环境数或 writer

当前基线：auto 的位姿 25.32%、渲染 20.30%、物理推进 25.69%、RGB 复制 2.91%。

方案顺序：先去重复位姿与图像副本，再用设备追踪定位渲染或物理；传输重叠后置。

读图讲解：本版本的 TiledCamera 已是 Camera 别名；仅换类名没有新增并行收益。



当前计算：以 auto 的 142.71 秒 execute 为分母，假定目标成本线性缩放且其余不变。

支持范围：位姿 6→1 情景减少 21.1%，两项合计减少 22.6%；都是算术情景而非实测。

读图讲解：不能外推最终文件吞吐；该口径还有排空、合并以及并发窗口重叠。

下一轮：输出等价通过之后，才比较最终合格吞吐

单项消融与重复

A0 现有 auto
A1 只去重位姿刷新
A2 只复用 CPU RGB
A3 两项组合

4 环境、8 条成功目标
同 seed / 分辨率 / 压缩
至少 3 对交替顺序重复

逐帧正确性

完整 actions 与严格终态
frame_id 与 reset_epoch
重置 / 事件后的第一帧
两次 compute 的 history
CPU 帧不可变与归属
GPU / CPU 缓冲有界

图像容差来自基线重复

计时与设备诊断

记录主循环与最终就绪
分段闭合 / 内存峰值
主线程 CPU / 设备时间线

NVTX + CUDA / 图形追踪
先核对本地工具兼容性

profiler 与吞吐运行分开
以合格条数 / 分钟判断

当前状态：新增缓存、渲染和流水线方案均未实施；本页是下一轮实验协议。

验证要求：单项消融、交替顺序重复，核对动作、帧归属、重置观测、终态与内存峰值。

读图讲解：先证明输出等价再比较最终合格吞吐；诊断 profiler 与吞吐运行分开。

从观察到解释：每一步结论都对应证据

已观察到

v1 异步最终就绪更早
执行 / 等待阶段均值下降
后台每条仍保存约 27–28 秒
末态检查均为 16/16

这是本轮完整系统的结果

与代码一致的解释

保存转入 CPU 子进程
生成与保存可在时间上重叠
主流程减少压缩与写入负担
仍承担打包和预算等待

机制解释不等于贡献分解

仍待验证

每项改动各贡献多少？
重复运行排名是否稳定？
GPU kernel 是否相互重叠？

需要消融、重复与设备追踪
不能只从 GPU 利用率推断

当前实验：v1 最终就绪、阶段、writer 与设备采样；没有逐项消融或 kernel 追踪。

支持结论：整体效率改善已观察到；保存重叠是机制解释，独立因果贡献尚未分离。

读图讲解：逐层区分实测现象、代码解释与未验证推断，避免把相关性写成证明。

cuRobo 库级变化 ≠ 本次 AutoData 全部启用并验证

维度	库级能力变化	本次实验覆盖
GPU 并行	v1 已有多初值并行求解 v2 继续扩展 GPU 计算	每环境规划器；未实现跨环境批量请求 不把 CPU writer 当作 GPU 并行升级
轨迹优化	v1 已有平滑性目标 v2 强调 B-spline 与动力学约束	读取 v2 插值轨迹 未验证本任务力矩约束激活与满足
场景感知	v2 增强 GPU 原生 深度融合 TSDF / ESDF	从 USD 解析碰撞几何 没有深度融合建图对照
高自由度计算	v2 强调拓扑感知运动学 可微逆动力学与自碰撞计算	OpenArm 左臂任务 没有全身机器人性能对照
性能结论	官方结果对应其测试任务 不能直接转成生产吞吐收益	v2 完整保存 16 条、末态通过 15/16 跨日期单轮，尚未确立性能优势

当前依据：官方 v1 文档、v2 仓库与论文，以及本项目冻结适配代码。

支持结论：库具备的能力不等于本实验已启用验证；不引用库基准推算本项目提速。

读图讲解：先讲库级变化，再说明本任务实际使用范围；官方基准不是本任务速度。

接口迁移：目标类型、参数顺序、配置传递和轨迹语义均需适配

v1：当前适配器调用

配置构造
MotionGenConfig.load_from_robot_config(...)
MotionGenPlanConfig(...)

目标与调用
Pose
plan_single(start_state, goal_pose, plan_config)

轨迹结果
优先 optimized_plan，空时取插值
get_full_js → get_ordered_joint_state

v2：实验冻结适配器调用

配置构造
MotionPlannerCfg.create(...)
robot / scene_model / device_cfg

目标与调用
GoalToolPose.from_poses(...)
plan_pose(goals, start_state, max_attempts=...)

轨迹结果
get_interpolated_plan()
reorder(joint_names) → FK → 世界系位姿

上层保留 update_world_and_plan_motion(...) → bool；enable_retiming 默认值仍有差异，详见正文。

当前依据：本地 v1 适配器与实验冻结 v2_planner.py，详细行号见接口证据。

支持结论：迁移涉及配置、目标、调用与轨迹语义；保持上层任务契约不等于库 API 相同。

读图讲解：按配置、目标、调用和轨迹结果四步讲，强调不是仅替换 import。

cuRobo v2：保留 AutoData 任务契约，适配模型、规划与轨迹接口

输入适配

原机器人配置深拷贝
ee_link → tool_frames
retract_config →
default_joint_position

目标 / 物体 → 基座系
开闭夹爪参数原地更新

规划与接触

MotionPlanner.plan_pose
GoalToolPose 目标

搬运时附着物体
接触阶段临时调整碰撞
finally 恢复 / detach

输出适配

get_interpolated_plan
读取稠密执行轨迹

按 joint_names 重排
可选关节距离重采样
FK → 世界系末端位姿

8 个环境各有规划对象；不把样条控制点直接执行，也不把规划成功等同于最终数据合格。

当前依据：v2 适配器的模型配置、规划调用与轨迹输出接口。

支持结论：v2 已接入生产流程；接口适配本身不能证明速度或质量优于 v1。

读图讲解：沿输入、接触、输出三段说明适配，重点解释插值采样与关节顺序。

v2 补跑设计：补齐输出链，再讨论版本差异

为什么重跑

旧记录缺最终就绪与校验链
部分分片已不在原路径

旧记录保留为不完整证据
新输出使用独立目录
不覆盖旧实验状态

怎样运行与验收

冻结 launcher 与保留环境
同源输入，8 环境，seed 101
目标 16 条记录成功

生成 → 保存 → 排空 / 合并
最终文件读回 + 原阈值检查

哪些条件未控制

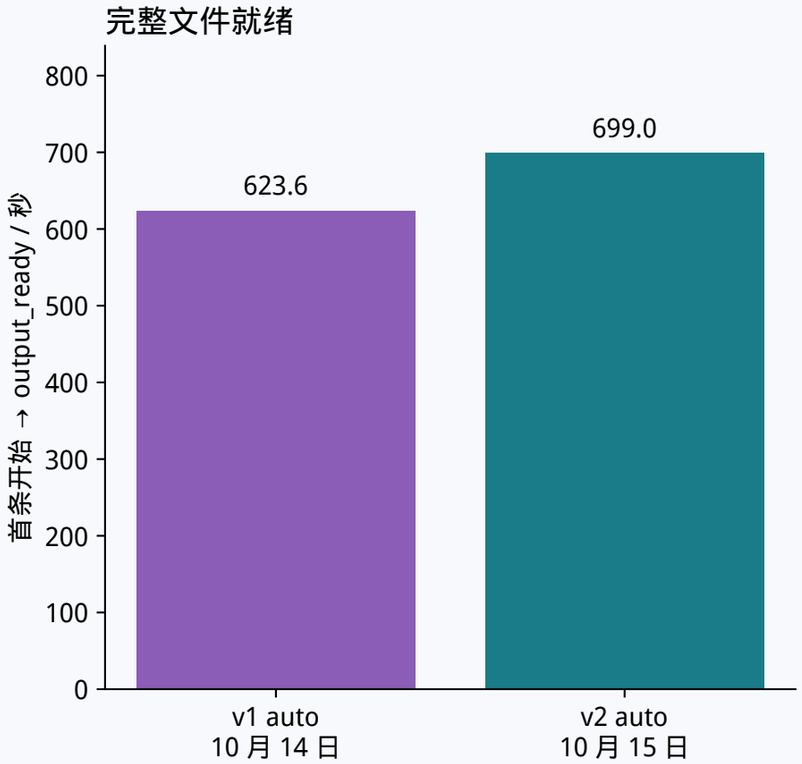
v1 与 v2 测试日期不同
缓存、热身与系统状态不同
完成 episode 分母不同

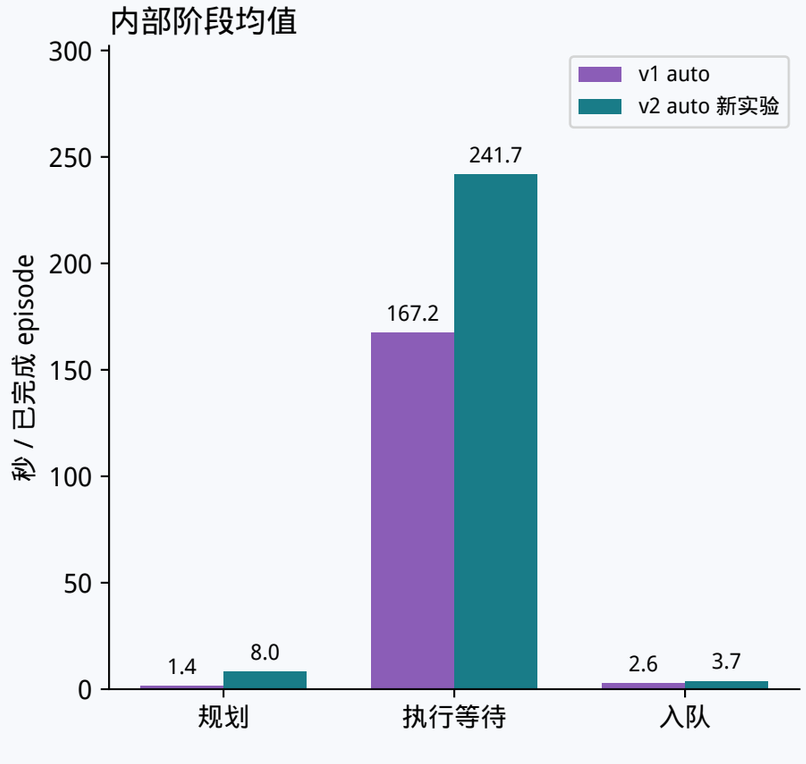
没有同日交错重复
因此不隔离纯后端因果效应

当前实验：冻结 v2 launcher，8 环境、seed 101、目标 16 条；独立输出与原阈值校验。

支持结论：补齐端到端输出证据；跨日期单轮与完成样本数差异限制版本因果比较。

读图讲解：先说明为什么补跑、如何完成和验收，再交代跨日期比较的限制。





新实验检查（末态未全通过）

生成成功：16 / 16
最终保存：16 条
末态通过：15 / 16
实际 writer：4
记录成功：1.373 条 / 分钟
末态合格：1.287 条 / 分钟

跨日期单轮观测，缓存 / 热身 / 系统状态未控制一致；不能将差值单独归因于版本升级。

---

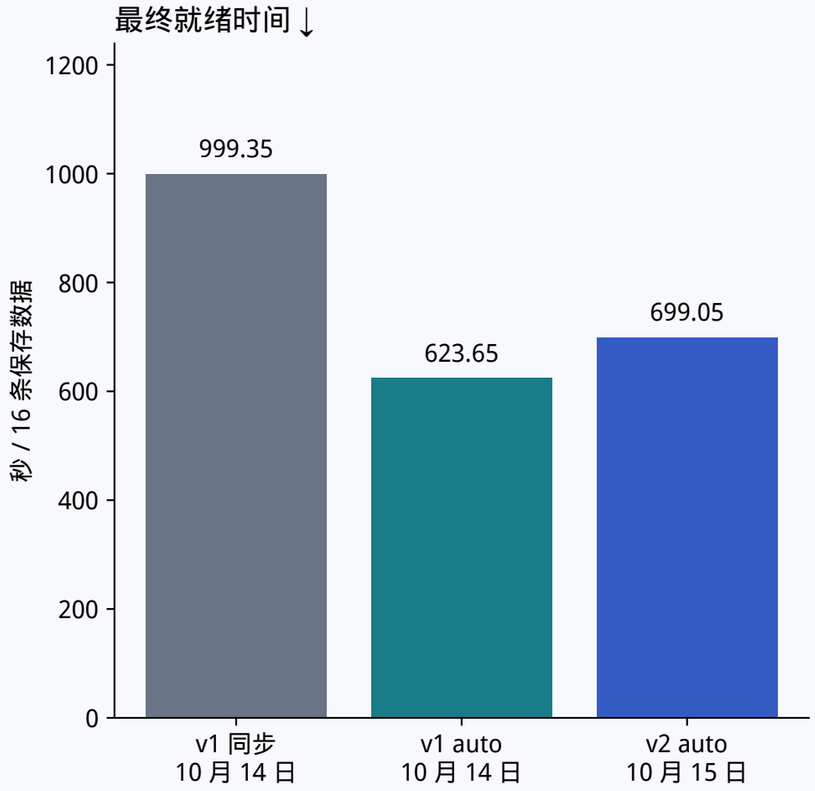
42 / 51 · 四、v2 迁移与实验

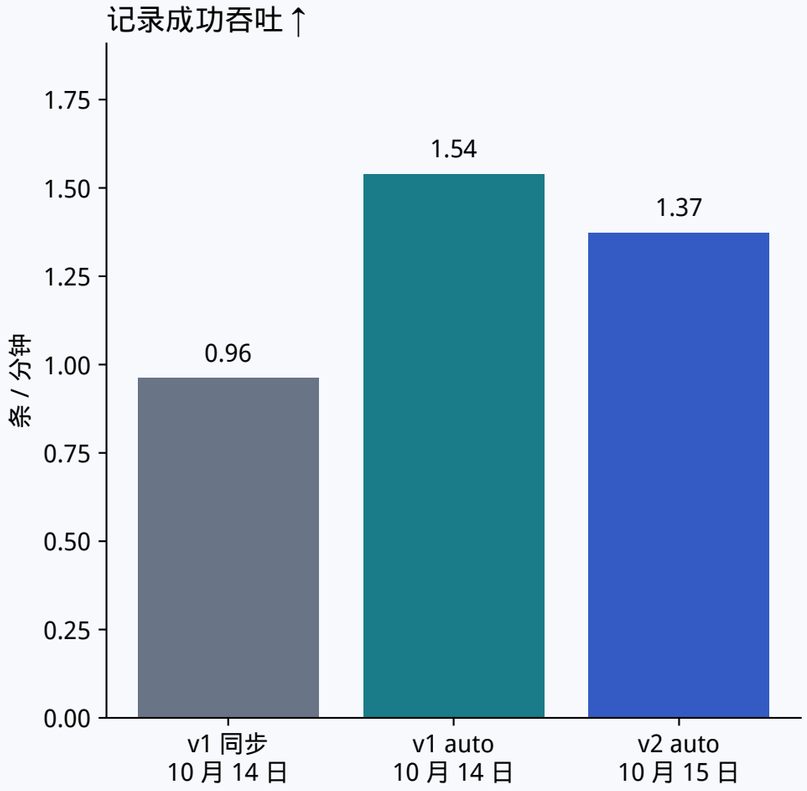
当前实验：v2 新重跑保存 16 条，最终就绪 699.05 秒，末态检查通过 15/16。

支持结论：补齐完整运行与输出校验，但质量未全通过；跨日期单轮不能隔离版本影响。

读图讲解：先读最终就绪与内部阶段，再读 15/16 末态通过；完成运行不等于全量合格。

实现改变不等于性能优势，需结合完整输出与质量





解释与质量边界

v1 auto 对比同步
耗时减少 37.6%
吞吐提高 60.2%
末态均通过 16/16

v2 auto：末态通过 15/16
完成接口迁移与整轮运行
尚未建立性能优势

当前实验：v1 同步与 auto 为同日单轮；v2 auto 为次日重跑，均保存 16 条。

支持结论：v1 auto 本轮效率提高；v2 末态仅 15/16 通过，尚不能确立其性能优势。

读图讲解：统一用 auto 比较保存策略；不要把跨日期差值全部归因于规划库。

结论：效率改进、接口迁移与质量验收分别陈述

已观察到的收益

v1 auto 对同步基线
最终就绪耗时减少 37.6%
记录成功吞吐增加 60.2%
末态检查均通过 16/16

本轮整体保存改造有效

已完成的 v2 工作

迁移规划模型和调用接口
正确读取插值轨迹并重排关节
完整生成并保存 16 条
完成输出读回与末态检查

末态通过 15/16，未全合格

不能推出的结论

不能保证 auto 总是最快
不能把吞吐收益当 GPU 加速比
不能声称 v2 已建立性能优势
不能把库全部能力当作已验证

所有结果保留单轮限制

当前证据：v1 四组完整结果与 v2 重跑输出；v2 末态检查仍为 15/16。

支持结论：v1 异步整体改善效率；v2 接入已完成，但性能优势与全量质量尚未建立。

读图讲解：将已观察收益、已完成适配与仍未验证的优势分别陈述。

下一轮验证：先规定问题和判据，再执行实验

性能稳定性

同日交错重复 v1 / v2
保持输入、种子与计时一致
记录热身和系统状态
报告中位数、波动与合格吞吐

本页为方案，尚未执行

收益来源

分别改变保存策略与 writer 上限
记录实际进程数、预算等待
设备追踪单独做诊断运行
测动作就绪、规划与 kernel

避免追踪开销污染吞吐对照

v2 质量失败

复核 demo_2 末态角速度
检查结束条件与稳定等待
修复另记版本并完整重跑
保留原阈值与本轮失败记录

末态验收与性能比较同时报告

当前状态：本页是待执行方案，不含新增实验结果。

验证目标：同日重复、保存侧消融、设备追踪与末态失败复核，各自回答不同问题。

读图讲解：按验证问题选择实验；计划中的步骤不能作为已经得到的结论。

改进前：生成主流程等待导出



改进后：生成与保存重叠



机制图，无时间比例。质量判定保持独立；异步提交完成不能代替最终文件就绪。

当前依据：同步导出与异步保存池的控制流；本页为机制示意。

支持结论：异步保存允许生成与保存重叠；图中箭头不表示实测时序或独立收益。

读图讲解：回看同步阻塞与后台重叠；箭头表达控制关系，没有时间比例。

保存机制的串并行对照：所有生产配置仍使用 8 个仿真环境

同步保存 · workers=0



同一主流程承担保存等待

异步单 writer



提交后可继续生成；保存侧仍只有一个执行者

异步多 writer / auto



生成与保存重叠；在途内存超预算时等待，最终仍需合并

控制流示意，无耗时比例。异步提交后生成可继续，后台保存同时推进；不是三个版本的实测时间线。

当前依据：同步、单 writer、多 writer 的保存流程；所有生产配置均为 8 环境。

支持结论：流水线重叠与保存侧多进程是两种机制；图中没有实测时间比例。

读图讲解：单 writer 也能形成流水线；多 writer 进一步提高保存侧处理容量。

看并行单位与资源边界，不能只看数量

单进程多环境批量步进

当前生成侧

8 个环境任务协作调度
↓
动作汇合后批量步进
↓
GPU 数值计算

共享事件循环与仿真对象
同步调用 / 动作屏障会等待

CPU 多进程保存

当前保存侧

主进程打包 episode
↓ 共享内存
多个 writer 独立压缩
↓
各写分片 → 排空 / 合并

生成与保存可重叠
竞争 CPU、内存与磁盘

使用 GPU 的多个生产进程

未测试的另一种方案

进程 A：仿真 + 规划
进程 B：仿真 + 规划
↓
共享 GPU 或绑定不同 GPU
↓
协调任务与输出

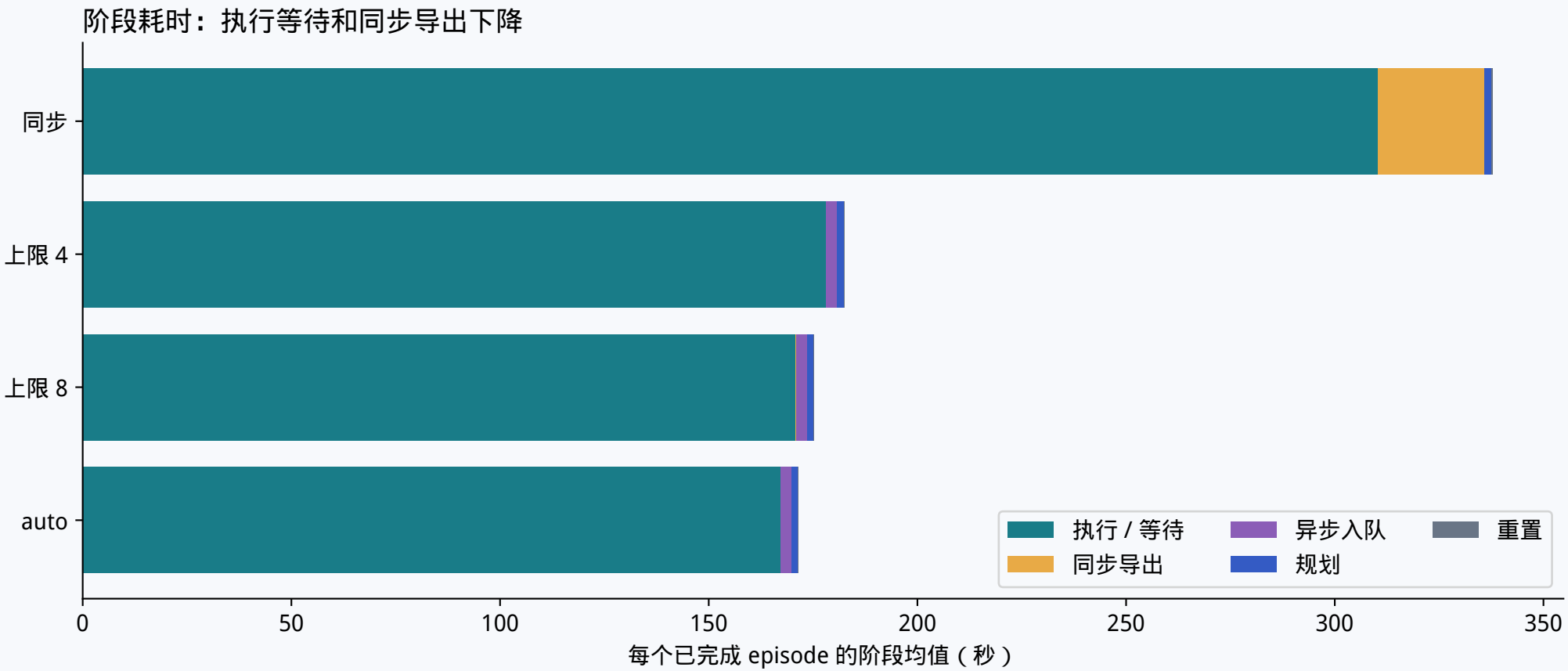
需额外设备资源与调度
进程数不等于加速倍数

设备利用率不能证明 kernel 重叠；当前没有单 GPU 多进程共享或多 GPU 部署的吞吐对照。

当前依据：现有单进程批量生成与 CPU 多进程保存的实现对照。

支持结论：应按执行单位区分并发与并行；单 GPU 多进程和多 GPU 部署尚无对照实验。

读图讲解：分清单进程批量生成、CPU 保存池和未测试的多 GPU 进程部署。

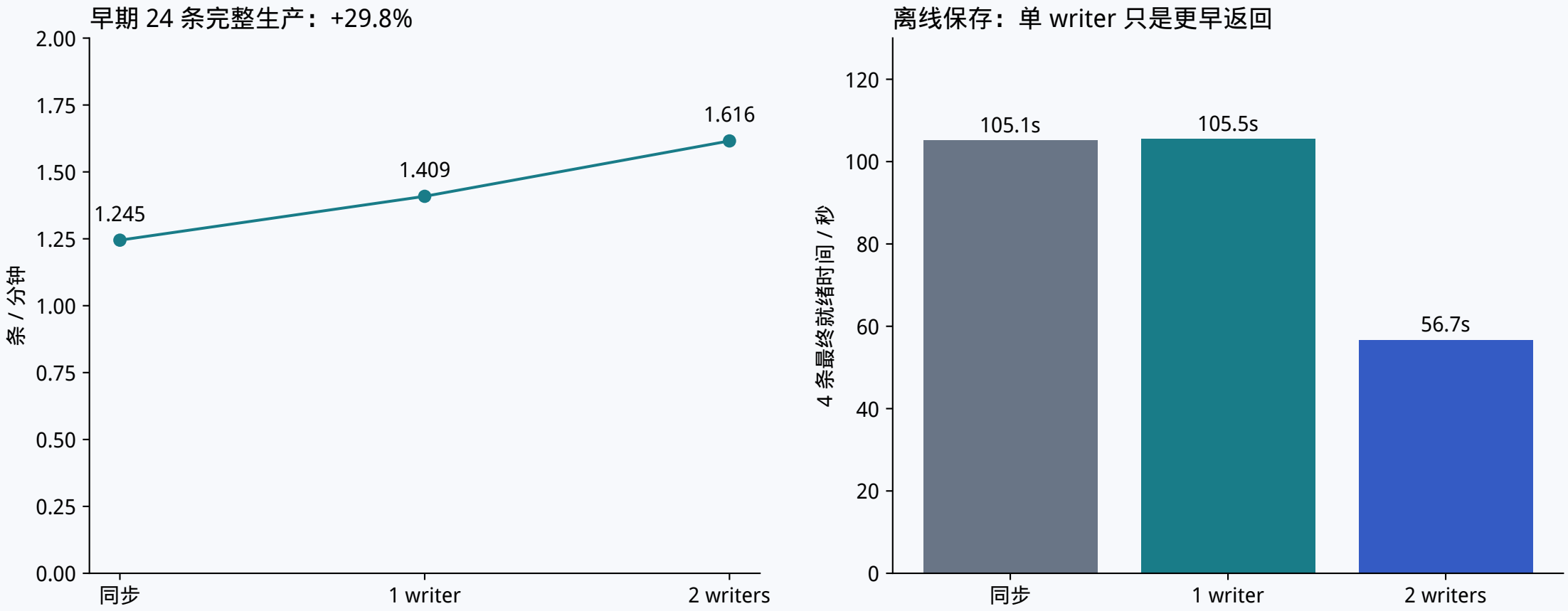


并发 episode 的耗时会重叠；此图不等于整批墙钟时间，也不覆盖所有未计时开销。

当前实验：各组 17 个完成 episode（含 1 次失败）的阶段均值。

支持结论：执行 / 等待和主流程导出负担下降；阶段均值不能相加为整批耗时。

读图讲解：用阶段组成理解单 episode 的负担；并发样本的时间会重叠。

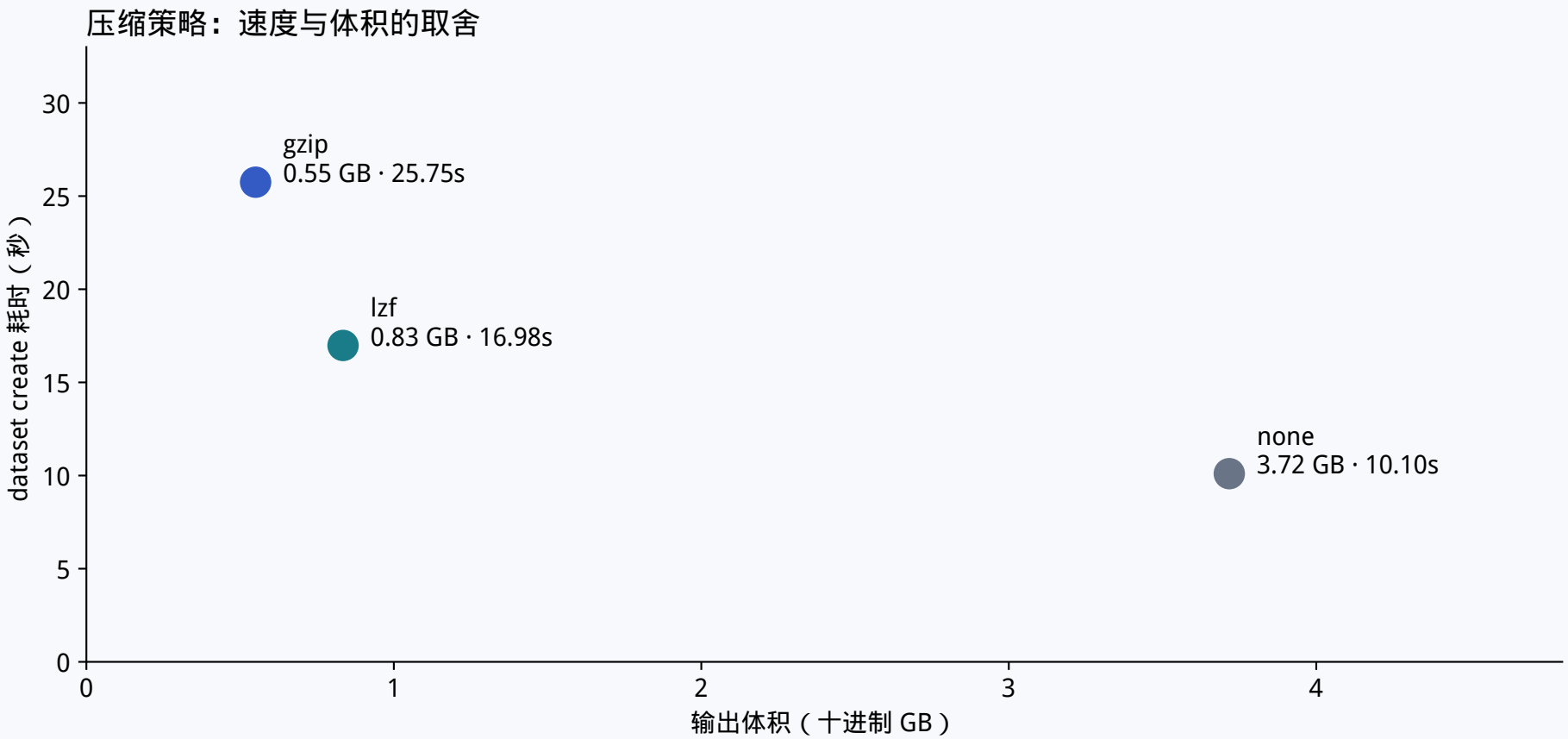


左：三个模式均为 24 条记录成功、23 条未态通过；右：CPU 离线回放，不能当作 GPU 生产吞吐。

当前实验：早期 24 条完整生产对照，以及 4 条数据的离线保存对照。

支持结论：双 writer 改善相应吞吐；离线单 writer 更早提交未缩短最终就绪时间。

读图讲解：先讲 24 条生产结果，再讲 4 条离线结果；两组不能合并计算平均加速比。



1342 帧 / 相机的离线 RGB 回放，保持 chunk 可比，单轮；含压缩、HDF5 和 I/O，非完整生产。

当前实验：每相机 1342 帧 RGB 离线回放，固定 chunk，比较三种压缩方式。

支持结论：保存耗时与文件体积存在取舍；结果不能直接替代完整生产吞吐。

读图讲解：横轴读体积，纵轴读保存耗时；压缩实验是后续取舍依据，不是已上线优化。